হাতে-কলমে সিরিজ · বই ২



নিজের AI Agent হাতে-কলমে

Groq আর Ollama দিয়ে ধাপে ধাপে বানাও তোমার প্রথম AI Agent

Ifte Samul Ohy · iftesamulohy.com

Licensed to sakilorb@gmail.com · Do not redistribute

সূচিপত্র

ভূমিকা: এই বইটা কেন, আর তোমার কী কী লাগবে 3

অধ্যায় ১: Chatbot আর Agent — পার্থক্যটা কোথায়? 6

অধ্যায় ২: Groq — তোমার ফ্রি সুপারফাস্ট ইঞ্জিন 10

অধ্যায় ৩: প্রথম API Call — ৫ লাইনে "হ্যালো" 14

অধ্যায় ৪: Terminal Chatbot — যে বাংলায় কথা বলে 19

অধ্যায় ৫: তোমার PC কি পারবে? (Configuration Check) 24

অধ্যায় ৬: Ollama — তোমার নিজের Computer-এ AI 28

অধ্যায় ৭: Tool Calling — LLM-কে হাত-পা দেওয়া 33

অধ্যায় ৮: আসল AI Agent — Think, Act, Observe, Repeat 40

অধ্যায় ৯: Agent যখন ঘুমায় না — ২৪/৭ চালানোর দরজা 50

শেষ অধ্যায়: এরপর কী? 53

--- 55

Appendix: বিপদে পড়লে এই পাতা (Troubleshooting) 56

এক টাকাও খরচ না করে বানাও নিজের AI Agent

Licensed to sakilorb@gmail.com · Do not redistribute

ভূমিকা: এই বইটা কেন, আর তোমার কী কী লাগবে

রাত এগারোটা। তুমি ফোন scroll করছ, আর ফিডে একের পর এক ভেসে আসছে — কেউ AI দিয়ে নিজের business-এর হিসাব automate করে ফেলেছে, কেউ AI-কে দিয়ে সারাদিনের email-এর summary বানাচ্ছে, কেউ আবার এমন একটা program বানিয়েছে যেটা নিজে নিজে সিদ্ধান্ত নিয়ে কাজ করে। দেখতে দেখতে মাথায় দুটো চিন্তা আসে:

এক — "ইশ, আমিও যদি এরকম কিছু বানাতে পারতাম!"

দুই — "কিন্তু এসব করতে নিশ্চয়ই অনেক টাকা লাগে। দামি GPU লাগে, paid API লাগে, ডলারে বিল দিতে হয়..."

প্রথম চিন্তাটা একদম ঠিক। দ্বিতীয়টা সম্পূর্ণ ভুল। আর সেই ভুলটা ভাঙানোর জন্যই এই বই।

এই বইয়ের promise

এই বইটা শেষ করলে তোমার নিজের computer-এ চলবে:

- একটা **chatbot** — যেটা বাংলায় কথা বলে, আগের কথা মনে রাখে, আর তুমি নিজে code লিখে বানিয়েছ
- একটা **আসল AI Agent** — যেটাকে একটা কাজ দিলে সে নিজে ভেবে, ধাপে ধাপে, নিজের সিদ্ধান্তে কাজটা শেষ করে
- আর bonus হিসেবে — তোমার computer-এ, internet ছাড়াই, সম্পূর্ণ নিজস্ব একটা AI

মোট খরচ: **শূন্য টাকা।** কোনো credit card না, কোনো paid subscription না, কোনো "free trial শেষ হলে টাকা কাটবে" ফাঁদ না। যা যা ব্যবহার করব সব সত্যিকারের free — কীভাবে সম্ভব, সেটাও বইয়ের ভেতরে ব্যাখ্যা করা আছে।

"হাতে-কলমে" সিরিজের দ্বিতীয় বই

এটা আমার "হাতে-কলমে" সিরিজের দ্বিতীয় বই। প্রথম বই **"কন্টেক্সট ইঞ্জিনিয়ারিং হাতে-কলমে"**-তে শিখেছিলে AI-এর সাথে কথা বলার কৌশল — কীভাবে প্রশ্ন সাজালে AI-এর কাছ থেকে সেরা উত্তর বের করা যায়।

এই বইয়ে আমরা এক ধাপ এগোব। এবার AI-এর সাথে শুধু কথা বলব না — AI-কে দিয়ে **কাজ করাব।**

আগের বইটা পড়া থাকলে কিছু জিনিস চেনা চেনা লাগবে, ভালো লাগবে। আর পড়া না থাকলেও কোনো সমস্যা নেই — এই বই সম্পূর্ণ স্বয়ংসম্পূর্ণ, শূন্য থেকে শুরু করলেই চলবে।

এই বই যা শেখাবে না (আগেই পরিষ্কার করে নিই)

বন্ধুর মতো সোজা কথা বলি — কারণ hype বেচা এই বইয়ের কাজ না:

- এই বই তোমাকে **Machine Learning শেখাবে না।** আমরা কোনো model train করব না, কোনো গণিতের সমীকরণ কষব না। আমরা তৈরি model ব্যবহার করব — ঠিক যেমন গাড়ি চালানো শিখতে engine বানানো শিখতে হয় না।
- এই বই পড়ে **সাত দিনে "AI Engineer" হয়ে যাবে না।** ওরকম promise যারা করে, তাদের থেকে দূরে থেকো।

Licensed to sakilorb@gmail.com · Do not redistribute

- এই বইয়ের agent দিয়ে **রাতারাতি টাকা income হবে না।** তবে যে foundation পাবে, সেটার ওপর দাঁড়িয়ে সামনে অনেক কিছু বানাতে পারবে — সে রাস্তাটা শেষ অধ্যায়ে দেখাব।

যা promise করছি শুধু সেটাই: বই শেষে তোমার হাতে একটা চলন্ত, কাজ-করা AI Agent থাকবে, আর সেটা কীভাবে কাজ করে তা তুমি ভেতর থেকে বুঝবে।

কী কী লাগবে (checklist)

শুরুর আগে মিলিয়ে নাও — লক্ষ করো, তালিকার সবকিছু free:

#	যা লাগবে	বিস্তারিত
১	একটা computer	Windows, Linux বা Mac — যেকোনোটা। ৮GB RAM হলে খুব ভালো, 8GB হলেও রাস্তা আছে (বইয়ে বলা আছে)
২	Internet connection	ধীরগতির হলেও চলবে — slow internet-এর জন্য আলাদা tips আছে Appendix-এ
৩	একটা email address	ফ্রি API account খোলার জন্য। Credit card **লাগবে না**
৪	Python-এর basic ধারণা	Variable, list, function, `if`, `for` loop — এগুলো চিনলেই যথেষ্ট
৫	কয়েক ঘণ্টা সময় আর একটু ধৈর্য	এটাই আসল investment

চার নম্বরটা নিয়ে একটু বলি — তোমাকে Python-এর master হতে হবে না। এই বইয়ের প্রতিটা code আমি লাইন ধরে ব্যাখ্যা করব। তবে একদম শূন্য থেকে শুরু করলে মাঝে মাঝে হোঁচট খেতে পারো — সেটা স্বাভাবিক, আর কোথায় সাহায্য পাবে তাও প্রাসঙ্গিক জায়গায় বলে দেব।

বইটা কীভাবে পড়বে

দুটো নিয়ম মানলে এই বই থেকে দশগুণ বেশি পাবে:

এক — code নিজের হাতে type করো। Copy-paste করলে চোখ পড়ে, কিন্তু হাত শেখে না। Type করার সময় ছোট ছোট ভুল হবে, সেই ভুল ঠিক করতে গিয়েই আসল শেখাটা হবে। (হ্যাঁ, আমি জানি তুমি তাও copy-paste করবে। অন্তত গুরুত্বপূর্ণ code-গুলো type কোরো।)

দুই — "নিজে করো" skip কোরো না। প্রতিটা অধ্যায়ের শেষে একটা ছোট challenge আছে। পাঁচ-দশ মিনিটের কাজ, কিন্তু ওই পাঁচ মিনিটেই concept-টা তোমার নিজের হয়ে যায়।

Licensed to sakilorb@gmail.com · Do not redistribute

নিজে করো ০

. ওপরের checklist-টা মিলিয়ে নাও — সব আছে তো?

. তোমার computer-এ একটা নতুন folder বানাও, নাম দাও `ai-agent` । এই বইয়ের সব code এই folder-এ থাকবে।

ব্যস, তুমি ready। চলো শুরু করি।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ১: Chatbot আর Agent — পার্থক্যটা কোথায়?

দোকানের দুই কর্মচারীর গল্প

একটা মুদি দোকানের কথা ভাবো। দোকানে দুইজন কর্মচারী কাজ করে — রফিক আর সুমন।

রফিক দাঁড়ায় counter-এ। তুমি জিজ্ঞেস করলে, "চিনি আছে?" রফিক বলবে, "আছে, ৫৫ টাকা কেজি।" জিজ্ঞেস করলে, "মিনিকেট চাল কেমন?" সে বলবে, "ভালো, নতুন চালান এসেছে।" রফিকের জ্ঞান আছে, উত্তর দিতে পারে চমৎকার — কিন্তু তার কাজ এটুকুই। তুমি প্রশ্ন করবে, সে উত্তর দেবে। প্রশ্ন না করলে সে চুপচাপ দাঁড়িয়ে থাকবে।

সুমন-এর ব্যাপার আলাদা। দোকানদার তাকে সকালে বলে দিলেন, "গোডাউন থেকে নতুন মাল এনে তাকগুলো সাজিয়ে রাখো, আর কী কী আনলে খাতায় হিসাব তুলে রেখো।" — এই এক লাইন বলেই দোকানদার অন্য কাজে চলে গেলেন।

এবার সুমন কী করে দেখো:

. প্রথমে সে **ভাবল** — আগে গোডাউনে গিয়ে দেখি কী কী মাল এসেছে।

. তারপর **কাজ করল** — গোডাউনে গেল।

. গিয়ে **দেখল** — চাল আর ডাল এসেছে, কিন্তু চিনির বস্তা আসেনি।

. আবার **ভাবল** — চিনি নেই যখন, আপাতত চাল-ডাল দিয়েই শুরু করি, চিনির কথাটা খাতায় note করে রাখি।

. আবার কাজ, আবার দেখা, আবার ভাবা... যতক্ষণ না পুরো কাজটা শেষ হয়।

দুপুরে দোকানদার ফিরে দেখলেন — তাক সাজানো, হিসাব খাতায় তোলা, আর খাতার কোণায় লেখা: "চিনি আসেনি, supplier-কে ফোন দিতে হবে।"

এই দুইজনের পার্থক্যটাই এই পুরো বইয়ের বিষয়।

রফিক হলো chatbot। প্রশ্ন → উত্তর। ব্যস।

সুমন হলো agent। একটা লক্ষ্য → নিজে ভেঙে, ধাপে ধাপে, নিজের সিদ্ধান্তে → কাজ শেষ।

তুমি ChatGPT বা Claude-এর সাথে যখন কথা বলো, তখন সেটা মূলত রফিক-mode: তুমি লেখো, সে উত্তর দেয়। দারুণ উত্তর দেয়, কিন্তু নিজে থেকে তোমার computer-এর একটা file-ও খুলে দেখে না। আর এই বইয়ে আমরা বানাব সুমন — যাকে কাজ দিয়ে তুমি চা খেতে যেতে পারবে।

আচ্ছা, এই LLM জিনিসটা আসলে কী?

Agent বানানোর আগে তার brain-টা চিনে নেওয়া দরকার। Agent-এর brain হলো **LLM** — Large Language Model। ChatGPT, Claude, Llama, Gemini — এরা সবাই LLM।

ভয়ের কিছু নেই, ভেতরের গণিত আমাদের লাগবে না। মূল ধারণাটা একটা লাইনে বলা যায়:

Licensed to sakilorb@gmail.com · Do not redistribute

> **LLM হলো এমন একটা program, যেটা পৃথিবীর বিশাল পরিমাণ লেখা পড়ে একটা কাজ শিখেছে — "এরপরের শব্দটা কী হবে" সেটা আন্দাজ করা।**

শুনতে সামান্য লাগছে? একটা খেলা খেলি। নিচের বাক্যটা মনে মনে শেষ করো:

"আকাশের রং ___"

তুমি প্রায় নিশ্চিতভাবে ভাবলে "নীল"। কীভাবে পারলে? কারণ জীবনভর তুমি এই শব্দগুলো এই সাজানোতে দেখেছ। LLM-ও ঠিক এটাই করে — তবে কোটি কোটি পৃষ্ঠা পড়ার অভিজ্ঞতা নিয়ে, আর এক শব্দ না, পুরো উত্তর ধরে।

মজার ব্যাপার হলো, "পরের শব্দ আন্দাজ করা"-র মতো সাদামাটা একটা কাজ যখন বিশাল scale-এ করা হয়, তখন যেটা বেরিয়ে আসে সেটা দেখতে অনেকটা বুদ্ধিমত্তার মতো — সে ব্যাখ্যা করতে পারে, code লিখতে পারে, বাংলায় কবিতাও লিখে ফেলে। সে কি সত্যি সত্যি "বোঝে"? — এটা দার্শনিকদের তর্কের বিষয়। আমরা practical মানুষ; আমাদের জন্য যেটা জরুরি: **জিনিসটা কাজ করে, আর আমরা এটাকে কাজে লাগাতে পারি।**

কিন্তু LLM-এর একটা বড় সমস্যা আছে

LLM যত brilliant-ই হোক, তার একটা সীমাবদ্ধতা আছে যেটা বুঝলেই "agent কেন দরকার" পরিষ্কার হয়ে যায়:

LLM শুধু লেখা নিতে পারে, আর লেখা দিতে পারে। ব্যস — এইটুকুই।

- সে ঘড়ি দেখতে পারে না — জিজ্ঞেস করলে হয় বলবে "আমি জানি না", নয়তো আন্দাজে ভুল সময় বলবে
- সে তোমার computer-এর কোনো file খুলতে পারে না
- সে internet ঘেঁটে আজকের খবর আনতে পারে না
- সে কোনো button চাপতে, email পাঠাতে, কিছু করতে পারে না

ভাবো — একজন অসম্ভব জ্ঞানী পরামর্শদাতা, কিন্তু তিনি ঘরবন্দি। জানালা নেই, দরজা নেই, হাত-পা নেই। তুমি চিরকুট পাঠালে তিনি চিরকুটে উত্তর দেন। জ্ঞান আছে টইটম্বুর, কিন্তু নিজে এক পা-ও নড়তে পারেন না।

এই ঘরবন্দি জ্ঞানীকে যদি আমরা **হাত-পা** দিই (যাতে তিনি কাজ করতে পারেন), আর **লেগে থাকার একটা নিয়ম** দিই (যাতে এক ধাপে না থেমে কাজ শেষ পর্যন্ত যান) — তাহলেই তিনি রফিক থেকে সুমন হয়ে যান।

Agent-এর সূত্র: তিনটা শব্দে পুরো বই

তাহলে দাঁড়াল:

> **Agent = LLM + Tools + Loop**

- **LLM** → brain। ভাবার, বোঝার, সিদ্ধান্ত নেওয়ার কাজটা করে।
- **Tools** → হাত-পা। File পড়া, হিসাব করা, সময় দেখা — বাস্তব কাজগুলো করার ক্ষমতা।

Licensed to sakilorb@gmail.com · Do not redistribute

- **Loop** → লেগে থাকা। ভাবো → কাজ করো → ফলাফল দেখো → আবার ভাবো... কাজ শেষ না হওয়া পর্যন্ত।

সুমনের গল্পটা আবার মনে করো — সে ঠিক এই loop-টাই চালাচ্ছিল: ভাবল, গোডাউনে গেল (কাজ), চিনি নেই দেখল (ফলাফল), plan বদলাল (আবার ভাবা)। AI জগতে এই চক্রটার একটা নাম আছে: **think → act → observe → repeat।** এই চারটা শব্দ মনে রেখো — অধ্যায় ৮-এ আমরা এটাকে নিজের হাতে code-এ লিখব।

আর এই সূত্রটাই পুরো বইয়ের route map:

সূত্রের অংশ	কোথায় শিখবে
LLM (brain জোগাড়)	অধ্যায় ২–৬: ফ্রি cloud API (Groq) আর নিজের computer-এ local AI (Ollama)
Tools (হাত-পা লাগানো)	অধ্যায় ৭: LLM-কে function চালাতে দেওয়া
Loop (পুরো agent)	অধ্যায় ৮: সব জোড়া লাগিয়ে আসল AI Agent
Bonus	অধ্যায় ৯: agent-কে ২৪/৭ চালানোর দরজা

চোখের সামনে একটা agent (শুধু কল্পনায়, আপাতত)

Code লেখার আগে একবার মনে মনে দেখে নিই ব্যাপারটা কেমন দেখাবে। ধরো বইয়ের শেষ দিকে গিয়ে তুমি তোমার agent-কে বললে:

"আমার notes folder-এ যা যা file আছে, সবগুলোর একটা summary report বানাও।"

Agent ভেতরে ভেতরে যা করবে:

```
ভাবনা:  আগে দেখি folder-এ কী কী আছে।
কাজ:    list_files("notes")
দেখলাম: তিনটা file — class-note.txt, todo.txt, idea.md

ভাবনা:  প্রথমটা পড়ে ফেলি।
কাজ:    read_file("class-note.txt")
দেখলাম: [file-এর লেখা...]

ভাবনা:  এটার summary মনে রাখলাম। পরেরটা পড়ি।
...
ভাবনা:  সব পড়া শেষ। এবার report লিখে দিই।
কাজ:    write_file("report.md", "...")
দেখলাম: লেখা হয়ে গেছে।  কাজ শেষ!
```

খেয়াল করো — কোন file আগে পড়বে, কখন থামবে, শেষে কী করবে — **প্রতিটা সিদ্ধান্ত agent নিজে নিচ্ছে।** তুমি শুধু লক্ষ্যটা বলে দিয়েছ। এটাই chatbot আর agent-এর আসল পার্থক্য, আর ঠিক এই জিনিসটাই তুমি অধ্যায় ৮-এ নিজের হাতে বানাবে।

এই অধ্যায়ে কোনো code নেই — ইচ্ছে করেই। ভিত মজবুত তো বাড়ি মজবুত। পরের অধ্যায় থেকে হাত চলবে keyboard-এ।

Licensed to sakilorb@gmail.com · Do not redistribute

নিজে করো ১

খাতা-কলম নাও (বা phone-এর notes app)। তোমার জীবনের এমন **একটা বিরক্তিকর, বারবার করতে হয় এমন কাজ** লেখো, যেটা তুমি একটা agent-কে দিয়ে করাতে চাও।

যেমন কেউ হয়তো লিখবে: "প্রতিদিন সকালে কয়েকটা news site দেখে গুরুত্বপূর্ণ খবরগুলোর summary চাই", কেউ লিখবে: "Downloads folder-টা category অনুযায়ী নিজে নিজে গুছিয়ে ফেলুক।"

লিখে রাখো, হারিয়ো না। বইয়ের শেষ অধ্যায়ে এই লেখাটাই হবে তোমার প্রথম নিজস্ব agent project-এর নীলনকশা।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ২: Groq — তোমার ফ্রি সুপারফাস্ট ইঞ্জিন

আগে ইঞ্জিন, পরে গাড়ি

আগের অধ্যায়ে সূত্রটা দাঁড় করিয়েছি: **Agent = LLM + Tools + Loop**। তো প্রথম কাজ — একটা LLM জোগাড় করা। মানে আমাদের agent-এর brain।

ব্যাপারটা অনেকটা গাড়ি বানানোর মতো। বডি, চাকা, স্টিয়ারিং — সব পরে; আগে দরকার একটা ইঞ্জিন। আর আমাদের ভাগ্য ভালো — ২০২৬ সালে দাঁড়িয়ে পৃথিবীর অন্যতম দ্রুত AI ইঞ্জিন পাওয়া যাচ্ছে **সম্পূর্ণ ফ্রি-তে।** নাম: **Groq।**

কিন্তু নামটা নিয়ে একটা গোলমাল আছে, আর সেটা আগে না মেটালে অনেকে ভুল জায়গায় গিয়ে account খুলে বসে থাকবে। তাই শুরুতেই —

Groq ≠ Grok — দুটো সম্পূর্ণ আলাদা জিনিস!

নামে মাত্র এক অক্ষরের পার্থক্য, কিন্তু company দুটোর কোনো সম্পর্ক নেই:

	Groq (আমরা এটা ব্যবহার করব)	**Grok (এটা না)**
বানান	G-r-o-**q**	G-r-o-**k**
কী জিনিস	Fast AI inference API company	xAI (Elon Musk)-এর chatbot
Website	**groq.com** / console.groq.com	grok.com
আমাদের কাজে লাগবে?	হ্যাঁ	না

মনে রাখার সহজ trick: আমাদেরটা **q** দিয়ে শেষ — আর **q** মানে **quick**। Groq-এর পুরো ব্যবসাই speed নিয়ে, তাই "quick"-এর q।

Account খোলার সময় ঠিকানা লিখবে: **console.groq.com** — Google-এ "grok api" search করে ভুল দরজায় ঢুকে যেও না!

Groq কী, আর এত fast কেন?

Groq একটা আমেরিকান company, যাদের ব্যবসা হলো — অন্যদের বানানো open-source LLM (যেমন Meta-র Llama series) **অবিশ্বাস্য গতিতে চালিয়ে** API হিসেবে দেওয়া। মানে ওরা নিজেরা model বানায় না; ওরা model চালানোর ওস্তাদ।

গতির রহস্যটা হলো ওদের নিজস্ব chip — **LPU (Language Processing Unit)**। সাধারণত AI চালানো হয় GPU দিয়ে, যেটা আসলে বানানো হয়েছিল game-এর graphics-এর জন্য; AI চালানো তার side business। আর LPU বানানোই হয়েছে শুধু

Licensed to sakilorb@gmail.com · Do not redistribute

একটা কাজের জন্য — language model চালানো। ফলাফল? যেখানে সাধারণ API-তে উত্তর আসে টাইপরাইটারের গতিতে, Groq-এ উত্তর আসে প্রায় ঝপ করে — সেকেন্ডে শত শত শব্দ।

প্রথমবার নিজের code থেকে Groq-এর response দেখলে তোমার মনে হবে কোথাও চিটিং হচ্ছে। হচ্ছে না — এটাই LPU।

আমাদের জন্য Groq বেছে নেওয়ার তিনটা কারণ:

. **সত্যিকারের free tier** — credit card লাগে না, শুধু email। "Free trial" না, আসলেই free tier।

. **Speed** — শেখার সময় প্রতি experiment-এ ৩০ সেকেন্ড অপেক্ষা করা vs ২ সেকেন্ড অপেক্ষা করা — সারাদিনে এটা বিশাল পার্থক্য।

. **OpenAI-compatible API** — এটার মানে পরের অধ্যায়ে বুঝবে, কিন্তু এক কথায়: Groq-এর জন্য লেখা code প্রায় না বদলেই অন্য অনেক provider-এ চলে। আমরা কোনো একটা company-র খাঁচায় বন্দি হচ্ছি না।

চলো account খুলি (ধাপে ধাপে)

পুরো প্রক্রিয়া ২-৩ মিনিটের। সাথে সাথে করে ফেলো:

ধাপ ১ — সঠিক ঠিকানায় যাও। Browser-এ লেখো: `console.groq.com` (আবার মনে করিয়ে দিই — **q** দিয়ে, k দিয়ে না!)

ধাপ ২ — Sign up করো। Email দিয়ে অথবা Google/GitHub account দিয়ে — যেটা সুবিধা। কোনো credit card, কোনো phone verification-এর ঝামেলা নেই।

`[SCREENSHOT: console.groq.com-এর signup পেজ]`

ধাপ ৩ — API Key বানাও। Login করার পর বাঁ দিকের menu-তে (বা dashboard-এ) **"API Keys"** নামে একটা section পাবে। সেখানে গিয়ে **"Create API Key"** button-এ click করো। Key-র একটা নাম দিতে বলবে — দাও `amar-prothom-agent` বা যা খুশি।

`[SCREENSHOT: API Keys পেজ, Create API Key button]`

ধাপ ৪ — Key-টা কপি করে রাখো, এক্ষুনি। Key বানানোর সাথে সাথে একটা লম্বা অক্ষর-সংখ্যার string দেখাবে, অনেকটা এরকম: `gsk_a B c ...`। **এই জানালাটা বন্ধ করলে key-টা আর কখনো পুরোটা দেখা যাবে না** — তাই এখনই copy করে নিরাপদ কোথাও রাখো (আপাতত একটা text file-এ রাখলেও চলবে, তবে পরের অধ্যায়ে এর চেয়ে ভালো ব্যবস্থা শেখাব)।

API Key মানে কী, আর কেন এটা গোপন জিনিস

API key হলো তোমার **digital চাবি**। তোমার code যখন Groq-কে বলবে "এই প্রশ্নের উত্তর দাও", সাথে এই key-টাও পাঠাবে — যাতে Groq বুঝতে পারে অনুরোধটা তোমার account থেকে আসছে।

আর চাবি যেমন কাউকে দিয়ে দাও না, API key-ও না:

- **কখনো** Facebook post, screenshot, বা public জায়গায় key দেখিও না

Licensed to sakilorb@gmail.com · Do not redistribute

- **কখনো** GitHub-এ code upload করার সময় key সহ upload কোরো না (এটা এত common ভুল যে এর জন্য আলাদা ব্যবস্থা শেখাব পরের অধ্যায়ে)
- Key ফাঁস হয়ে গেছে মনে হলে console.groq.com-এ গিয়ে পুরোনোটা delete করে নতুন বানাও — free, এক মিনিটের কাজ

Free tier-এ কতটুকু পাবে?

এবার সবচেয়ে জরুরি প্রশ্ন — free মানে কতটা free?

Groq-এর free tier-এ সীমা আছে, তবে শেখার জন্য যথেষ্টর চেয়ে বেশি। এই বই লেখার সময় পর্যন্ত চিত্রটা মোটামুটি এরকম:

	বড় model (যেমন Llama 3.3 70B)	ছোট model (যেমন Llama 3.1 8B)
প্রতি মিনিটে request	~৩০টা	~৩০টা
প্রতিদিন request	~১,০০০টা	~১৪,৪০০টা
দৈনিক token limit	আছে (সীমিত)	তুলনায় অনেক বেশি

দুটো নতুন শব্দ এখানে:

- **Request** = তোমার code একবার Groq-কে প্রশ্ন পাঠানো মানে একটা request। দিনে ১,০০০ request মানে — শেখার সময় তুমি সারাদিন experiment করেও এই সীমায় পৌঁছাবে না বললেই চলে।
- **Token** = LLM-এর জগতে লেখা মাপার একক। মোটা হিসাবে একটা token ≈ একটা শব্দের তিন-চতুর্থাংশ। প্রশ্ন যত বড়, উত্তর যত বড় — তত বেশি token খরচ। বড় model-এ request-এর পাশাপাশি দিনে মোট কত token ব্যবহার করলে, তারও একটা সীমা আছে — খুব লম্বা লম্বা লেখা নিয়ে কাজ করলে এটাতেই আগে ধাক্কা খাবে।

> **Disclaimer:** এই সংখ্যাগুলো **এই বই লেখার সময় পর্যন্ত** সঠিক। Free tier-এর limit company-গুলো সময়ে সময়ে বদলায়। হালনাগাদ সংখ্যা সবসময় পাবে Groq-এর official documentation-এ (console-এর ভেতরে "Rate Limits" section)। সংখ্যা বদলালেও মূল কথাটা বদলায়নি বহুদিন: **শেখার জন্য free tier-ই যথেষ্ট।**

আর limit পার হয়ে গেলে কী হবে? ভয়ের কিছু না — টাকা কাটবে না (কাটার উপায়ই নেই, card তো দাওনি!)। শুধু Groq একটা error ফেরত দেবে (নাম `: Too Many Requests`), মানে "একটু দম নাও ভাই"। কিছুক্ষণ পর আবার কাজ করবে। এই error দেখলে কী করবে, বিস্তারিত আছে Appendix-এ।

Licensed to sakilorb@gmail.com · Do not redistribute

কোন model দিয়ে শুরু করব?

Groq-এ ঢুকলে দেখবে অনেকগুলো model-এর নাম — Llama, Qwen, Whisper, আরও কত কী। আপাতত মাথা ঘামিয়ো না; এই বইয়ে আমরা মূলত দুটো ব্যবহার করব:

- `llama- . - b-versatile` — বড়, বুদ্ধিমান। জটিল কাজ, বাংলা ভাষা, agent-এর সিদ্ধান্ত নেওয়া — এসবে এটা ব্যবহার করব।
- `llama- . - b-instant` — ছোট, বিদ্যুৎগতি, দৈনিক limit অনেক বেশি। টুকটাক experiment-এর জন্য।

নামের সংখ্যাগুলো (B, B) মানে model-টার আকার — B মানে billion parameters। আপাতত এভাবে ভাবো: B হলো অভিজ্ঞ সিনিয়র, B হলো চটপটে জুনিয়র। কখন কাকে ডাকবে, সেটা কাজ বুঝে — আর এই বুদ্ধিটাই বই জুড়ে ধীরে ধীরে তৈরি হবে।

নিজে করো ২

. **console.groq.com**-এ account খোলো (বানানটা শেষবারের মতো মনে করিয়ে দিলাম)

. একটা API key বানাও আর নিরাপদ জায়গায় copy করে রাখো

. Console-এর ভেতরে একটু ঘুরে দেখো — বিশেষ করে **Playground** section-টা। ওখানে code ছাড়াই browser-এ model-গুলোর সাথে কথা বলে দেখতে পারো। Llama . B-কে বাংলায় কিছু জিজ্ঞেস করে দেখো তো — উত্তরের গতিটা খেয়াল কোরো!

Key হাতে? তাহলে তুমি ready। পরের অধ্যায়ে এই key দিয়ে জীবনের প্রথম API call — মাত্র ৫ লাইনের code-এ।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৩: প্রথম API Call — ৫ লাইনে "হ্যালো"

প্রথম চিরকুট

অধ্যায় ১-এর ঘরবন্দি জ্ঞানীকে মনে আছে? বলেছিলাম — তুমি চিরকুট পাঠালে তিনি চিরকুটে উত্তর দেন। এতদিন তুমি সেই চিরকুট পাঠিয়েছ ChatGPT বা Claude-এর chat box-এ টাইপ করে। আজ থেকে পাঠাবে **নিজের লেখা Python code দিয়ে।**

পার্থক্যটা বিশাল। Chat box-এ তুমি একজন ব্যবহারকারী। আর code দিয়ে পাঠালে তুমি একজন নির্মাতা — কারণ code একবার লিখলে সেটা হাজারবার চলতে পারে, অন্য code-এর সাথে জুড়তে পারে, আর শেষমেশ একটা agent-এর ভেতরের অংশ হয়ে যেতে পারে।

আর মজার খবর: প্রথম চিরকুটটা পাঠাতে লাগবে মাত্র **৫ লাইন code!** চলো মঞ্চটা সাজিয়ে ফেলি।

মঞ্চ সাজানো: project folder আর virtual environment

ভূমিকার "নিজে করো ০"-তে যে `ai-agent` folder বানিয়েছিলে, terminal (Windows-এ Command Prompt বা PowerShell) খুলে সেই folder-এ ঢোকো:

```
cd ai-agent
```

এবার বানাব একটা **virtual environment**। এটা কী জিনিস? ভাবো — তোমার computer-টা একটা বড় বাসা, আর প্রতিটা Python project-এর জন্য আলাদা আলাদা ঘর। Virtual environment হলো এই project-এর নিজস্ব ঘর — এখানে যা install করবে তা শুধু এই ঘরেই থাকবে, বাসার অন্য কিছু নোংরা করবে না।

Linux / Mac:

```
python    -m venv venv
source venv/bin/activate
```

Windows (Command Prompt / PowerShell):

```
python -m venv venv
venv\Scripts\activate
```

Activate হলে terminal-এর লাইনের শুরুতে `(venv)` লেখা দেখবে — মানে তুমি এখন এই project-এর নিজের ঘরে ঢুকে গেছ। এখন থেকে এই বইয়ের কোনো code চালানোর আগে দেখে নিও `(venv)` আছে কি না; না থাকলে আবার activate command-টা চালিয়ে নিও।

Licensed to sakilorb@gmail.com · Do not redistribute

> Terminal-এ এই যে দুটো command লিখলে — এটাই কিন্তু developer জীবনের আসল দরজা। Terminal-কে যদি এখনো একটু ভয়ের জিনিস মনে হয়, চিন্তা নেই — এই বইয়ের জন্য যতটুকু লাগে সব হাত ধরে দেখাব। আর terminal-টাকে একেবারে পোষ মানিয়ে ফেলতে চাইলে আমার Linux বইটা পড়ে দেখতে পারো: **লিনুকে পোষ মানানো** → https://iftesamulohy.com/products/linuke-posh-manano/

দুটো জিনিস install

`(venv)` চালু অবস্থায় লেখো:

```
pip install openai python-dotenv
```

- `openai` — LLM API-র সাথে কথা বলার library
- `python-dotenv` — API key নিরাপদে রাখার library (একটু পরেই কাজে লাগবে)

দাঁড়াও দাঁড়াও — আমরা না Groq ব্যবহার করব? তাহলে `openai` install করছি কেন?!

OpenAI-compatible মানে কী: এক চার্জারে সব ফোন

কয়েক বছর আগে প্রতিটা ফোন brand-এর আলাদা চার্জার ছিল — Nokia-র এক পিন, Samsung-এর আরেক। ভোগান্তির শেষ ছিল না। তারপর এল USB-C — এখন এক cable-এ প্রায় সব ফোন চলে।

LLM জগতে ঠিক এটাই ঘটেছে। OpenAI সবার আগে জনপ্রিয় API বানিয়েছিল, তাই ওদের `openai` Python library-টা হয়ে গেছে জগতের **USB-C**। এখন Groq সহ অনেক provider নিজেদের API এমনভাবে বানায় যেন এই একই library দিয়ে কাজ করা যায় — একে বলে **OpenAI-compatible API**।

তোমার জন্য এর মানে: `openai` library-কে শুধু বলে দিতে হবে, "ভাই, চিঠিটা OpenAI-এর ঠিকানায় না, Groq-এর ঠিকানায় পাঠাও" — ব্যস। এই "ঠিকানা"-টার technical নাম `base_url`। আজ Groq, কাল Ollama (অধ্যায় ৬-এ দেখবে — সেখানেও এই একই code চলবে, শুধু ঠিকানা বদলাবে!) — এক শেখাতেই সব দরজা খোলা।

প্রথম code: ৫ লাইনে "হ্যালো"

`ai-agent` folder-এ একটা নতুন file বানাও — নাম `hello.py`। তারপর লেখো (মনে আছে তো — নিজের হাতে type!):

Licensed to sakilorb@gmail.com · Do not redistribute

```
from openai import OpenAI

client = OpenAI(api_key="এখানে_তোমার_Groq_API_key", base_url="https://api.groq.com/openai/
v  ")

response = client.chat.completions.create(
    model="llama-  .  -    b-versatile",
    messages=[{"role": "user", "content": "বাংলায় আমাকে হ্যালো বলো আর এক লাইনে উৎসাহ দাও!"}]
)

print(response.choices[  ].message.content)
```

`এখানে_তোমার_Groq_API_key` জায়গায় আগের অধ্যায়ে বানানো তোমার আসল key-টা বসাও (quotation-এর ভেতরে)। তারপর terminal-এ:

```
python hello.py
```

এক-দুই সেকেন্ড... আর তোমার screen-এ বাংলায় ভেসে উঠল AI-এর উত্তর।

থামো এক মিনিট। যা ঘটল সেটা আরেকবার ভাবো — **তোমার লেখা code**, তোমার computer থেকে, পৃথিবীর আরেক প্রান্তের একটা ৭০ বিলিয়ন parameter-এর AI-এর সাথে কথা বলে এল। আর এতে খরচ হলো শূন্য টাকা। এই ৫ লাইনই সেই ভিত, যার ওপর বাকি পুরো বই দাঁড়িয়ে।

Code-টা লাইন ধরে বুঝি

- **লাইন ১:** `openai` library থেকে `OpenAI` নামের জিনিসটা আনলাম।
- **লাইন ২ (client):** Groq-এর সাথে সংযোগের ব্যবস্থা। দুটো তথ্য দিলাম — তোমার পরিচয় (`api_key`) আর ঠিকানা (`base_url`)। এই `client` -ই এখন থেকে আমাদের ডাকপিয়ন।
- **লাইন ৩-৬ (response):** আসল চিরকুট পাঠানো। `model` -এ বললাম কোন AI-কে চাই, আর `messages` -এ দিলাম আমাদের বার্তা। `messages` -এর ভেতরের গঠনটা (`role` , `content`) এখন একটু অদ্ভুত লাগতে পারে — পরের অধ্যায়ে এটাই আমাদের মূল বিষয়, তখন পানির মতো পরিষ্কার হবে।
- **শেষ লাইন:** Groq একটা বড়সড় উত্তরের প্যাকেট (`response`) ফেরত পাঠায়। তার ভেতর থেকে আসল লেখাটুকু বের করলাম — `response.choices[  ].message.content` । প্যাকেটের বাকি অংশে আরও কিছু মজার তথ্য থাকে, যেমন `response.usage` -এ কত token খরচ হলো। কৌতূহল হলে `print(response.usage)` লিখে দেখে নিও।

এক্ষুনি একটা বদভ্যাস বন্ধ করি: key কখনো code-এ না

`hello.py` কাজ করেছে, দারুণ। কিন্তু ওই file-টার একটা বড় সমস্যা আছে — **API key-টা সরাসরি code-এর ভেতরে বসানো।** এটাকে বলে hardcoding, আর এটা এমন এক বদভ্যাস যেটার কারণে প্রতিদিন হাজার হাজার মানুষের key ফাঁস হয় — কেউ code-এর screenshot share করে, কেউ ভুলে GitHub-এ upload করে দেয়।

Licensed to sakilorb@gmail.com · Do not redistribute

নিয়মটা প্রথম দিনেই রক্তে ঢুকিয়ে নাও: **গোপন জিনিস থাকবে আলাদা file-এ, code-এ থাকবে শুধু তার উল্লেখ।**

`ai-agent` folder-এ `.env` নামে একটা file বানাও (হ্যাঁ, নামের শুরুতেই ডট — extension নেই)। ভেতরে শুধু এক লাইন:

```
GROQ_API_KEY=তোমার_আসল_key_এখানে
```

(এখানে quotation লাগবে না, `=` -এর দুপাশে space-ও না।)

এবার `hello.py` -কে বদলে ফেলো এই final রূপে:

```
import os
from dotenv import load_dotenv
from openai import OpenAI

    .env file থেকে গোপন তথ্যগুলো load করলাম
load_dotenv()

    client বানালাম — key এখন আর code-এ নেই, .env থেকে আসছে
client = OpenAI(
    api_key=os.getenv("GROQ_API_KEY"),
    base_url="https://api.groq.com/openai/v  "
)

response = client.chat.completions.create(
    model="llama-  .  -    b-versatile",
    messages=[{"role": "user", "content": "বাংলায় আমাকে হ্যালো বলো আর এক লাইনে উৎসাহ দাও!"}]
)

print(response.choices[  ].message.content)
```

আবার `python hello.py` চালাও — একই ফলাফল, কিন্তু এবার code file-টা নিরাপদ। এখন এই file কাউকে দেখালে, share করলে, এমনকি GitHub-এ দিলেও তোমার key ফাঁস হবে না — শুধু খেয়াল রাখবে `.env` file-টা যেন কখনো share না হয়। (GitHub ব্যবহার করলে `.gitignore` file-এ `.env` লিখে রাখো — তাহলে ভুলেও upload হবে না।)

এই বইয়ের বাকি সব code এই `.env` পদ্ধতিই ব্যবহার করবে।

কিছু একটা গেল ভুল? (quick checklist)

প্রথম API call-এ error খাওয়া একদম স্বাভাবিক — লজ্জার কিছু না। সবচেয়ে common তিনটা:

- `AuthenticationError` / → key ভুল আছে বা `.env` load হয়নি। `.env` file-টা কি `hello.py` -র একই folder-এ আছে? Key copy-paste-এ শুরু-শেষে space ঢুকে যায়নি তো?
- `ModuleNotFoundError: No module named 'openai'` → `(venv)` activate করা ছাড়াই চালিয়েছ, অথবা install হয়নি। Activate করে আবার `pip install openai python-dotenv` দাও।
- **Internet-related error / timeout** → connection দেখো; মাঝে মাঝে আবার চেষ্টা করলেই হয়ে যায়।

Licensed to sakilorb@gmail.com · Do not redistribute

এগুলোসহ আরও অনেক common সমস্যার সমাধান একসাথে পাবে বইয়ের শেষে **Appendix**-এ — আটকে গেলে ওটাই তোমার প্রথম গন্তব্য।

নিজে করো ৩

1. `hello.py` -তে model-এর নাম বদলে দাও: `llama-3.3-70b-versatile` → `llama-3.1-8b-instant`। আবার চালাও। **গতির পার্থক্যটা** টের পাচ্ছ? এই "কোন কাজে কোন model" অনুভূতিটাই আসল শিক্ষা।
2. `content` -এর লেখাটা বদলে নিজের একটা প্রশ্ন করো — বাংলায়। যেমন: "বাংলাদেশের তিনটা দর্শনীয় জায়গার নাম বলো" বা "Python শেখার তিনটা tips দাও"।
3. Bonus: `print(response.usage)` যোগ করে দেখো তোমার প্রশ্ন-উত্তরে কত token খরচ হলো। অধ্যায় ২-এর দৈনিক token limit-এর সাথে মিলিয়ে হিসাব করো — এই গতিতে দিনে কতগুলো প্রশ্ন করতে পারবে?

৫ লাইন থেকে শুরু করেছিলে — এখন তোমার হাতে নিরাপদ, নিজের মতো বদলানো যায় এমন একটা AI-চালিত program। পরের অধ্যায়ে একে বানাব সত্যিকারের কথা-মনে-রাখা chatbot।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৪: Terminal Chatbot — যে বাংলায় কথা বলে

যে বন্ধু প্রতিবার তোমাকে ভুলে যায়

আগের অধ্যায়ের `hello.py` দিয়ে একটা ছোট experiment করা যাক। File-টা দু'বার চালাও — প্রথমবার `content` -এ লেখো:

"আমার নাম রাকিব। মনে রেখো।"

Model খুব ভদ্রভাবে বলবে, "অবশ্যই রাকিব, মনে রাখব!" এবার `content` বদলে লেখো:

"আচ্ছা বলো তো, আমার নাম কী?"

চালাও। আর দেখো — model হয় বলবে "আমি তো জানি না", নয়তো আন্দাজে একটা নাম বানিয়ে বলবে। এই মাত্র যে কথা দিল, সেটাই বেমালুম ভুলে গেছে!

কারণটা অধ্যায় ১-এই বলেছিলাম, এবার হাড়ে হাড়ে টের পেলে: **LLM-এর নিজের কোনো স্মৃতি নেই।** প্রতিটা API call তার কাছে জীবনের প্রথম কথোপকথন। ChatGPT-র মতো app-গুলো যে "মনে রাখে" বলে মনে হয়, সেটা model-এর ক্ষমতা না — সেটা app-এর কারসাজি। আর আজ আমরা সেই কারসাজিটাই নিজের হাতে বানাব।

কারসাজিটা কী: ডাক্তারের চেম্বারের নিয়ম

আমাদের দেশের ডাক্তারের চেম্বারের দৃশ্যটা ভাবো। ডাক্তার প্রতিদিন শত শত রোগী দেখেন — তোমাকে আলাদা করে মনে রাখা অসম্ভব। তাহলে চিকিৎসা চলে কীভাবে? **তুমি প্রতিবার পুরোনো prescription-এর file-টা সাথে নিয়ে যাও।** ডাক্তার দুই মিনিটে file উল্টে দেখে নেন — আগে কী হয়েছিল, কী ওষুধ চলছে — তারপর এমনভাবে কথা বলেন যেন সবই মনে আছে।

Chatbot-এর memory ঠিক এই নিয়মেই চলে:

> **প্রতিবার নতুন প্রশ্ন পাঠানোর সময়, আগের পুরো কথোপকথনটাও সাথে পাঠিয়ে দাও।**

Model প্রতিবার পুরো file পড়ে নেয়, তারপর এমনভাবে উত্তর দেয় যেন সব মনে আছে। রোগী হিসেবে file টানার দায়িত্বটা যেমন তোমার, chatbot-এ সেই দায়িত্বটা তোমার code-এর। আর এই "file"-টাই হলো আগের অধ্যায়ে দেখা সেই `messages` list।

`messages` আর তিনটা role: কথোপকথনের খাতা

`messages` হলো একটা Python list — কথোপকথনের খাতা, যেখানে প্রতিটা কথা একটা করে entry। প্রতিটা entry-তে দুটো জিনিস: কে বলল (`role`) আর কী বলল (`content`)। Role হয় তিন রকম:

Licensed to sakilorb@gmail.com · Do not redistribute

Role	কে	কাজ
`system`	পরিচালক	খাতার একদম শুরুতে থাকে। Model-কে বলে দেয় সে কে এবং কীভাবে কথা বলবে। ব্যবহারকারী এটা দেখে না
`user`	তুমি	তোমার প্রতিটা প্রশ্ন/কথা
`assistant`	Model	Model-এর প্রতিটা উত্তর

একটা তিন-লাইনের কথোপকথন খাতায় যেমন দেখায়:

```
messages = [
    {"role": "system", "content": "তুমি একজন বন্ধুসুলভ বাংলাভাষী সহকারী।"},
    {"role": "user", "content": "আমার নাম রাকিব। মনে রেখো।"},
    {"role": "assistant", "content": "অবশ্যই রাকিব! মনে থাকবে।"},
    {"role": "user", "content": "আমার নাম কী?"}
]
```

এই পুরো list-টা পাঠালে model শেষ প্রশ্নের উত্তরে ঠিকই বলবে — "তোমার নাম রাকিব!" কারণ উত্তরটা তার স্মৃতিতে নেই, **খাতাতেই লেখা আছে।**

এর মধ্যে `system` role-টা আলাদা করে চেনার মতো। এটা model-এর জন্য মঞ্চের পেছনের নির্দেশনা — "তুমি এই চরিত্র, এই ভাষায়, এই মেজাজে কথা বলবে।" আমার আগের বই "কন্টেক্সট ইঞ্জিনিয়ারিং হাতে-কলমে" পড়ে থাকলে চিনতে পারছ — সেখানে যে শিখেছিলে "AI-কে কী context দিচ্ছ তার ওপরই উত্তরের মান নির্ভর করে", এই `messages` খাতাটাই হলো সেই context-এর বাস্তব চেহারা। System prompt যত যত্নে লিখবে, chatbot তত ভালো আচরণ করবে।

এবার বানাই: `chatbot.py`

পুরো পরিকল্পনাটা সরল:

. খাতা খুলব, প্রথমে system prompt লিখব

. ব্যবহারকারীর কথা নেব → খাতায় লিখব → পুরো খাতা Groq-কে পাঠাব

. উত্তর দেখাব → উত্তরটাও খাতায় লিখব

. আবার ২ নম্বরে — যতক্ষণ না ব্যবহারকারী "exit" লেখে

`ai-agent` folder-এ নতুন file: `chatbot.py`

Licensed to sakilorb@gmail.com · Do not redistribute

```
import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()

client = OpenAI(
    api_key=os.getenv("GROQ_API_KEY"),
    base_url="https://api.groq.com/openai/v  "
)

    আমাদের chatbot-এর চরিত্র — এখানেই তার ব্যক্তিত্ব ঠিক হয়
SYSTEM_PROMPT = """তুমি 'বন্ধু' — একজন আন্তরিক বাংলাভাষী AI সহকারী।
সবসময় সহজ, প্রাঞ্জল বাংলায় উত্তর দেবে।
Technical শব্দ (যেমন API, code) English-এই রাখবে।
উত্তর অযথা লম্বা করবে না।"""

    কথোপকথনের খাতা — শুরুতে শুধু system prompt
messages = [{"role": "system", "content": SYSTEM_PROMPT}]

print("   'বন্ধু' চালু হলো! কথা শেষ হলে লিখো: exit")

while True:
            ১. ব্যবহারকারীর কথা নিই
    user_input = input("\nতুমি: ")

            exit লিখলে বিদায়
    if user_input.strip().lower() in ["exit", "quit", "bye"]:
        print("\nবন্ধু: আজ তাহলে আসি, আবার কথা হবে!   ")
        break

            ২. খাতায় লিখে পুরো খাতা Groq-কে পাঠাই
    messages.append({"role": "user", "content": user_input})

    response = client.chat.completions.create(
        model="llama-  .  -    b-versatile",
        messages=messages
    )

            ৩. উত্তর দেখাই এবং উত্তরটাও খাতায় লিখি
    reply = response.choices[  ].message.content
    print(f"\nবন্ধু: {reply}")
    messages.append({"role": "assistant", "content": reply})
```

চালাও:

```
python chatbot.py
```

এবার আসল test — সেই শুরুর experiment-টা আবার করো:

Licensed to sakilorb@gmail.com · Do not redistribute

```
তুমি:  আমার নাম রাকিব। মনে রেখো।
বন্ধু:  অবশ্যই রাকিব!  মনে থাকবে।

তুমি:  আমার নাম কী?
বন্ধু:  তোমার নাম রাকিব!
```

মনে রেখেছে! কারণ তুমি এখন জানো — জাদুটা model-এর মাথায় না, তোমার `messages` list-এ। মাত্র ৩০ লাইনের মতো code-এ তুমি এমন একটা জিনিস বানালে, যেটা কয়েক বছর আগেও science fiction ছিল — তোমার terminal-এ, বাংলায় কথা বলা, কথা-মনে-রাখা একটা AI।

> Code-এর `while True` , `.append()` , list-এর কারবার — এগুলো দেখে যদি মনে হয় "চলছে তো, কিন্তু কেন চলছে পুরোটা ধরতে পারছি না" — তাহলে Python-এর ভিতটা একটু পোক্ত করে নেওয়া ভালো। আমার **Journey by Python** বইটা ঠিক এই কাজের জন্যই লেখা — শূন্য থেকে ধাপে ধাপে, বাংলায়: https://iftesamulohy.com/products/journey-by-python/

খাতা যখন মোটা হয়ে যায়: token-এর হিসাব

একটা জিনিস খেয়াল করেছ? আমাদের নিয়মে প্রতিবার পুরো খাতা পাঠাতে হয়। কথা যত লম্বা হবে, খাতা তত মোটা — আর অধ্যায় ২-এর কথা মনে করো: প্রতিটা পাঠানো শব্দ মানে **token খরচ**। মোটা খাতার তিনটা সমস্যা:

- Token খরচ বাড়ে (free tier-এর দৈনিক token limit-এ আগে ধাক্কা)
- উত্তর আসতে একটু বেশি সময় লাগে
- খাতা খুব বেশি মোটা হলে model-এর পড়ার সীমা (context limit) পেরিয়ে error আসে

সমাধানের সবচেয়ে সরল কৌশল: **পুরোনো পাতা ছিঁড়ে ফেলা।** ডাক্তারের file-এও তো দশ বছর আগের কাগজ কেউ রাখে না — সাম্প্রতিকগুলোই আসল। আমরা রাখব system prompt (খাতার প্রথম পাতা — এটা ছেঁড়া যাবে না, নাহলে chatbot তার চরিত্রই ভুলে যাবে!) + শেষ ২০টা কথা।

`chatbot.py`-র `while` loop-এর একদম শেষে (শেষ `messages.append(...)` লাইনের পরে) এটুকু যোগ করো:

```
            খাতা বেশি মোটা হয়ে গেলে পুরোনো কথা ছেঁটে ফেলি
            প্রথম পাতা (system prompt) সবসময় রাখি + শেষ ২০টা কথা
    if len(messages) >       :
        messages = [messages[  ]] + messages[-    :]
```

`messages[  ]` হলো system prompt, আর `messages[-    :]` মানে list-এর শেষ ২০টা entry। শেখার পর্যায়ে এই সরল কৌশলই যথেষ্ট। (বড় production app-গুলো আরও চালাক পদ্ধতি ব্যবহার করে — পুরোনো কথার summary বানিয়ে রাখা, গুরুত্বপূর্ণ তথ্য আলাদা করে জমানো — সেই জগতের দরজাটা শেষ অধ্যায়ে দেখাব।)

Licensed to sakilorb@gmail.com · Do not redistribute

System prompt নিয়ে খেলা: এক code, হাজার chatbot

এখানেই মজার অংশ। `chatbot.py`-র বাকি সব একই রেখে শুধু `SYSTEM_PROMPT` বদলাও — সম্পূর্ণ নতুন chatbot পেয়ে যাবে:

```
SYSTEM_PROMPT = """তুমি একজন কড়া কিন্তু স্নেহশীল গণিত শিক্ষক।
ছাত্র প্রশ্ন করলে সরাসরি উত্তর না দিয়ে ছোট ছোট hint দেবে,
যেন ছাত্র নিজে উত্তরে পৌঁছায়। উত্তর সবসময় বাংলায়।"""
```

কিংবা:

```
SYSTEM_PROMPT = """তুমি বাংলাদেশের একজন অভিজ্ঞ রাঁধুনি।
যেকোনো প্রশ্নের উত্তরে দেশি রান্নার উদাহরণ টেনে আনবে।
মেজাজ সবসময় হাসিখুশি।"""
```

একই ৩০ লাইন code — একবার সে গণিত শিক্ষক, একবার রাঁধুনি। এবার নিশ্চয়ই বুঝতে পারছ, বাজারের অসংখ্য "AI app" আসলে ভেতরে কী — একটা LLM + একটা যত্নে লেখা system prompt + সুন্দর একটা চেহারা। সেই ভেতরের জিনিসটা এখন তোমার হাতের মুঠোয়।

নিজে করো ৪

. `SYSTEM_PROMPT` বদলে chatbot-কে সম্পূর্ণ নিজের একটা personality দাও। কিছু idea: যে সব উত্তর ছড়ায়/কবিতায় দেয় , যে নিজেকে ২০৫০ সালের AI ভাবে, বা যে প্রতিটা উত্তরের শেষে একটা করে fun fact বলে।

. তোমার chatbot-এর memory পরীক্ষা করো: শুরুতে তিনটা তথ্য বলো (নাম, প্রিয় খাবার, শখ), মাঝখানে ৪-৫টা অন্য কথা বলো, তারপর জিজ্ঞেস করো "আমার সম্পর্কে কী কী জানো?" — দেখো কতটা মনে রেখেছে।

. **Bonus (একটু কঠিন):** trimming-এর সংখ্যাটা / থেকে কমিয়ে / করে দাও, তারপর লম্বা কথোপকথন চালাও। দেখবে একসময় chatbot শুরুর তথ্যগুলো ভুলে যাচ্ছে — নিজের চোখে দেখো "স্মৃতি ছেঁটে ফেলা"-র প্রভাবটা কেমন। দেখা শেষে আবার ২১/২০-তে ফিরিয়ে রেখো।

তোমার chatbot এখন কথা বলে, মনে রাখে। কিন্তু এখনো সে অধ্যায় ১-এর "রফিক" — শুধু কথা, কাজ না। কাজে নামার আগে একটা জরুরি প্রশ্নের উত্তর দরকার: **তোমার computer নিজে AI চালাতে পারবে তো?** পরের অধ্যায়ে সেটাই মাপব।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৫: তোমার PC কি পারবে? (Configuration Check)

গাড়ি কেনার আগে engine দেখো

পরের অধ্যায়ে আমরা একটা দুর্দান্ত কাজ করব — **তোমার নিজের computer-এ, internet ছাড়া, একটা আস্ত AI চালাব।** কিন্তু তার আগে একটা সৎ প্রশ্নের মুখোমুখি হতে হবে: তোমার computer-টা পারবে তো?

ব্যাপারটা পুরোনো গাড়ি কেনার মতো। বাইরের রং দেখে কেনা বোকামি — চালাক ক্রেতা bonnet খুলে engine দেখে। আজ আমরা তোমার computer-এর bonnet খুলব। ভয় নেই — কোনো screwdriver লাগবে না, লাগবে কয়েকটা command। আর এই অধ্যায় শেষে তুমি নিশ্চিতভাবে জানবে ঠিক **কোন AI model তোমার machine-এ ভালো চলবে।**

Spoiler দিয়ে রাখি: উত্তরটা প্রায় সবসময় "হ্যাঁ, পারবে"। প্রশ্নটা "পারবে কি না" না — প্রশ্নটা "**কতটুকু** পারবে"।

তিনটা জিনিস মাপব — আর কেন

১. RAM — এটাই আসল হিরো। ভাবো, AI model হলো একটা মোটা বই, আর RAM হলো তোমার পড়ার টেবিল। বই পড়তে হলে পুরো বইটা টেবিলে রাখতে হবে — model চালাতে হলে পুরো model-টা RAM-এ ঢুকতে হবে। টেবিল ছোট হলে বইয়ের অর্ধেক মেঝেতে (মানে hard disk-এ) রেখে পড়তে হয় — তখন সব কিছু কচ্ছপের গতিতে চলে। তাই: **RAM যত, model তত।**

২. CPU cores — কর্মীর সংখ্যা। যত বেশি core, model তত দ্রুত কথা বলবে। ৪ core হলেও চলবে, ৮ হলে ভালো।

৩. GPU — থাকলে সোনায় সোহাগা, না থাকলেও সমস্যা নেই। সত্যি কথাটা বলে রাখি: আমাদের দেশের বেশিরভাগ student-এর laptop-এ আলাদা (dedicated) GPU নেই — আর এই বইয়ের পুরো পরিকল্পনা সেটা ধরেই করা। ছোট model CPU-তেই দিব্যি চলে, একটু ধীরে এই যা।

চলো এবার তোমার machine-এ এই তিনটা সংখ্যা বের করি। নিচে তিনটা OS-এর পদ্ধতিই আছে — **তোমারটা যেটা, শুধু সেই অংশটুকু করলেই হবে।** একটা কাগজে (বা phone-এর notes-এ) তিনটা সংখ্যা টুকে রেখো: RAM, core, GPU আছে/নেই।

Windows

সহজ রাস্তা — Task Manager: `Ctrl + Shift + Esc` চাপো → **Performance** tab।

- **Memory**-তে click → ডানে ওপরে মোট RAM (যেমন " . GB")
- **CPU**-তে click → নিচে "Cores" সংখ্যাটা
- বাঁ পাশের তালিকায় **GPU** নামে আলাদা entry থাকলে দেখো তার নাম — নামে "NVIDIA" বা "AMD Radeon" থাকলে তোমার dedicated GPU আছে; "Intel(R) UHD/Iris" জাতীয় কিছু থাকলে সেটা built-in graphics — আমাদের হিসাবে "GPU নেই" ধরো

Command-এর রাস্তা: Command Prompt খুলে (Start menu-তে `cmd` লিখে Enter):

Licensed to sakilorb@gmail.com · Do not redistribute

```
systeminfo | findstr /C:"Total Physical Memory"
echo %NUMBER_OF_PROCESSORS%
```

প্রথমটা RAM দেখাবে, দ্বিতীয়টা core (আসলে thread) সংখ্যা। আর GPU-র নাম জানতে PowerShell-এ:

```
Get-CimInstance Win    _VideoController | Select-Object Name
```

Linux

Terminal খোলো, তারপর:

```
free -h                          RAM — "Mem:" লাইনের "total" ঘরটা দেখো
nproc                              CPU core সংখ্যা — এক শব্দে উত্তর!
lscpu                              CPU-র পুরো কুণ্ডলী (model, core, speed সব)
lspci | grep -i vga             GPU-র নাম
```

ব্যস — চার লাইনে bonnet-এর ভেতরটা হাতের তালুতে। কোনো menu হাতড়ানো নেই, কোনো দশ click নেই।

Mac

সহজ রাস্তা: বাঁ-ওপরের → **About This Mac** → Memory আর Chip/Processor দুটোই এক পর্দায়।

Command-এর রাস্তা: Terminal-এ:

```
sysctl -n hw.memsize             RAM, byte হিসেবে (সংখ্যাটাকে                    দিয়ে ভাগ দিলে
GB)
sysctl -n hw.ncpu                   CPU core সংখ্যা
system_profiler SPHardwareDataType            পুরো hardware-এর বিবরণ একসাথে
```

Mac-দের জন্য একটা সুখবর: তোমার Mac যদি **Apple Silicon** (M , M , M ...) হয়, তাহলে তুমি এই অধ্যায়ের সবচেয়ে ভাগ্যবান পাঠক। M-series chip-এ RAM আর graphics একসাথে গাঁথা (unified memory) — local AI চালানোর জন্য এগুলো অসাধারণ machine।

Decision Table: তোমার সংখ্যা → তোমার model

কাগজে টুকে রাখা সংখ্যাগুলো এবার মেলাও। (Model-গুলোর নাম এখন অচেনা লাগলে চিন্তা নেই — পরের অধ্যায়ে সব চিনবে; আপাতত শুধু তোমার লাইনটা দাগিয়ে রাখো। Download size-টাও দিলাম, যেন internet-এর হিসাবটাও আগেই করতে পারো।)

Licensed to sakilorb@gmail.com · Do not redistribute

তোমার RAM	চালাবে এই model	Download size (আনুমানিক)	অভিজ্ঞতা কেমন হবে
৪ GB	`llama . : b`	~১.৩ GB	চলবে, ধীরে। ছোট কাজে ঠিক আছে
৮ GB	`llama .` (B) বা `qwen . : b`	~২ GB	**এই বইয়ের sweet spot** — শেখার সব কাজ আরামে হবে
১৬ GB+	`llama . : b` বা `qwen . : b`	~৪.৫–৫ GB	বেশ ভালো মানের উত্তর, ভদ্র গতি
+ dedicated GPU / Apple Silicon	ওপরের যেটাই চালাও	—	একই model, কয়েক গুণ গতি — bonus!

দুটো সৎ সতর্কবাণী:

- **RAM-এর পুরোটা model পাবে না।** ৮ GB RAM-এর machine-এ OS, browser, অন্য app মিলে এমনিতেই ৩-৪ GB খেয়ে বসে থাকে। তাই local AI চালানোর সময় **browser-এর পঞ্চাশটা tab বন্ধ করে নিও** — সত্যি বলছি, এটা রসিকতা না, এটাই সবচেয়ে কার্যকর optimization।
- টেবিলে নিজের লাইনের চেয়ে বড় model চালাতে গেলে কী হবে? Computer টা hang করবে, fan ঝড়ের বেগে ঘুরবে, আর উত্তর আসবে মিনিটে এক শব্দ। লোভ কোরো না — ছোট model ভালোভাবে চলা, বড় model-এর ধুঁকে ধুঁকে চলার চেয়ে হাজার গুণ ভালো।

আর যদি তোমার machine ৪ GB-রও নিচে হয়? মন খারাপ কোরো না — সেজন্যই তো অধ্যায় ২-৪ পুরোটা Groq দিয়ে সাজিয়েছি। Cloud-এর ৭০B model তো তোমার হয়েই আছে; পরের অধ্যায়টা তখন পড়ে রাখবে জ্ঞানের জন্য, হাতে-কলমে অংশটা Groq দিয়েই চালিয়ে যাবে। এই বইয়ের কোনো project-ই local AI ছাড়া আটকে থাকবে না।

একটা জিনিস খেয়াল করলে?

এই অধ্যায়ে তিনটা OS-এর পদ্ধতি পাশাপাশি দেখলে। Windows-এ Task Manager খোলো, tab খোঁজো, click করো... আর Linux-এ? `free -h` — ব্যস, উত্তর হাজির। `nproc` — উত্তর হাজির। যে প্রশ্নের জন্য অন্য জায়গায় menu হাতড়াতে হয়, Linux-এ সেটা এক লাইনের কথা।

এটা কোনো বিচ্ছিন্ন ঘটনা না — সামনের অধ্যায়গুলোতে বারবার দেখবে, developer-দের দুনিয়াটা আসলে terminal-এর দুনিয়া, আর terminal-এর রাজা Linux। আরও দুটো কথা এই ফাঁকে বলে রাখি:

- **তোমার সেই পুরোনো, ধুঁকতে থাকা laptop-টা?** Windows-এ যেটা চলতেই চায় না, Linux-এ সেটা দিব্যি প্রাণ ফিরে পায় — কারণ Linux নিজে RAM প্রায় খায়-ই না। মানে একই machine-এ AI model-এর জন্য বেশি RAM বেঁচে যায়। কম বাজেটের machine-এ local AI চালানোর সেরা চাল এটাই।
- **অধ্যায় ৯-এ যখন agent-কে ২৪/৭ চালানোর কথা বলব** — সেই server-দুনিয়ার প্রায় পুরোটাই Linux-এ চলে। আজকের এই ছোট্ট পরিচয়টা তখন কাজে দেবে।

Licensed to sakilorb@gmail.com · Do not redistribute

Terminal আর Linux-কে গোড়া থেকে, গল্পের ছলে পোষ মানাতে চাইলে আমার এই বইটা তোমার জন্যই লেখা: **লিনুকে পোষ মানানো** → https://iftesamulohy.com/products/linuke-posh-manano/ — তাড়া নেই; এই বই শেষ করে ধরলেও চলবে।

নিজে করো ৫

1. তোমার OS-এর পদ্ধতিতে তিনটা সংখ্যা বের করো: **RAM, CPU core, GPU আছে/নেই** — আর সেটা লিখে রাখো (পরের অধ্যায়ে লাগবেই)।
2. Decision table থেকে **তোমার model-টা** ঠিক করো, নামটা টুকে রাখো — download size সহ।
3. **Bonus:** পরিবারের বা বন্ধুর আরেকটা computer-এ একই check চালাও। দুটো machine-এর তুলনা করলে সংখ্যাগুলোর মানে আরও গভীরে ঢুকবে — আর কার machine-এ কোন model চলবে, সেটা বলে দিয়ে একটু ভাবও নিতে পারবে।

Machine চেনা শেষ, model বাছা শেষ। এবার সেই মুহূর্ত — যার জন্য এতক্ষণ মঞ্চ সাজালাম। পরের অধ্যায়ে তোমার computer-এ নামবে তোমার সম্পূর্ণ নিজস্ব AI — যেটা internet বন্ধ করে দিলেও কথা বলে যাবে।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৬: Ollama — তোমার নিজের Computer-এ AI

আজকের গন্তব্য: একটা অসম্ভবকে সম্ভব করা

এই অধ্যায়ের শেষে তুমি একটা কাজ করবে, যেটা কয়েক বছর আগে হলে মানুষ বিশ্বাসই করত না:

WiFi বন্ধ। Mobile data বন্ধ। Internet-এর সাথে কোনো সংযোগ নেই। তবুও তোমার computer-এ একটা AI দিব্যি তোমার সাথে কথা বলে যাচ্ছে।

কোনো company-র server না, কোনো API key না, কোনো rate limit না — সম্পূর্ণটাই তোমার machine-এ, তোমার নিয়ন্ত্রণে। ChatGPT ব্যবহার করা আর এটা করার মধ্যে পার্থক্যটা restaurant-এ খাওয়া আর নিজের রান্নাঘরে রাঁধতে শেখার মতো। চলো রান্নাঘরটা বানিয়ে ফেলি।

Ollama কী?

আগের অধ্যায়ে model-কে তুলনা করেছিলাম মোটা বইয়ের সাথে — বাস্তবে সেই "বই"-টা হলো একগাদা সংখ্যায় ভরা একটা বিশাল file। কিন্তু file তো আর নিজে নিজে চলে না — সেটাকে RAM-এ তুলে, প্রশ্ন খাওয়িয়ে, উত্তর বের করার একটা ব্যবস্থা লাগে। **Ollama** হলো সেই ব্যবস্থা — একটা free, open-source software, যেটা local AI চালানোর সব ঝক্কি (model নামানো, চালানো, manage করা) এক-দুইটা command-এ নামিয়ে এনেছে।

Ollama-কে ভাবতে পারো তোমার computer-এর **AI-এর দোকানদার**: তুমি বলবে "ওই model-টা দাও", সে নামিয়ে আনবে; বলবে "চালাও", সে চালিয়ে দেবে। ব্যস।

ধাপ ১: Ollama install

Windows / Mac: Browser-এ যাও **ollama.com/download** — নিজের OS-এর installer নামিয়ে, আর দশটা software-এর মতোই install করো। Next, Next, Finish।

Linux: এক লাইনেই কাজ শেষ:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Install ঠিকঠাক হয়েছে কি না দেখতে terminal-এ (Windows-এ Command Prompt) লেখো:

```
ollama --version
```

Version নম্বর দেখালেই — দোকানদার হাজির।

Licensed to sakilorb@gmail.com · Do not redistribute

ধাপ ২: Model নামানো — download-এর আগে পড়ো!

এবার আগের অধ্যায়ের homework কাজে লাগবে। Decision table থেকে তোমার যে model ঠিক করেছিলে, সেটাই নামাবে। ৮ GB RAM-ওয়ালাদের জন্য (আমাদের sweet spot):

```
ollama pull llama  .
```

`pull` মানে "নামিয়ে রাখো"। প্রায় ২ GB নামবে — আর এখানেই আমাদের দেশের বাস্তবতাটা সামনে আসে। তাই download শুরু করার আগে এই box-টা পড়ে নাও:

Slow internet-এ model নামানোর বুদ্ধি (BD edition)

- **হিসাব আগে:** তোমার model-এর size আগের অধ্যায়ের table-এ আছে। Mobile hotspot-এ নামালে data pack-এর হিসাবটা আগে করো — ২ GB model মানে সত্যি সত্যি ২ GB data। সীমিত pack থাকলে ছোট model (`llama  .  :  b` , ~১.৩ GB) দিয়ে শুরু করো — বইয়ের সব কাজ ওটাতেও চলবে।
- **মাঝপথে ছিঁড়ে গেলে ভয় নেই:** Ollama download **resume করতে পারে।** Connection কেটে গেলে বা `Ctrl+C` চাপলে — আবার একই `ollama pull` command দাও, যেখানে থেমেছিল মোটামুটি সেখান থেকেই ধরবে। পুরোটা আবার নামবে না।
- **রাতের রাস্তা:** বেশিরভাগ এলাকায় গভীর রাতে broadband-এর গতি ভালো থাকে। বড় model রাতে নামতে দিয়ে ঘুমিয়ে পড়ো — এটা যুগ যুগ ধরে চলা পরীক্ষিত দেশি কৌশল।
- **ভালো WiFi পেলেই কাজ সেরে রাখো:** University/office-এর দ্রুত WiFi-এর নাগাল পেলে model-টা তখনই নামিয়ে রেখো। সুখবর: **download জীবনে একবারই লাগে** — একবার নামলে model চিরকাল তোমার, আর কোনো data খরচ নেই।

ধাপ ৩: প্রথম কথা — আর সেই মুহূর্তটা

Download শেষ? এবার:

```
ollama run llama  .
```

কয়েক সেকেন্ড পর একটা prompt আসবে (`>>>`) — মানে model RAM-এ উঠে বসে আছে, তোমার কথার অপেক্ষায়। কিছু লেখো — উত্তর আসবে।

এবার। সেই মুহূর্ত।

WiFi বন্ধ করো। Mobile data বন্ধ করো। Airplane mode দিয়ে দাও, ইচ্ছে হলে router-এর তার-ও খুলে ফেলো।

তারপর আবার প্রশ্ন করো।

Licensed to sakilorb@gmail.com · Do not redistribute

...উত্তর আসছে। শব্দের পর শব্দ, তোমার নিজের computer-এর ভেতর থেকে। পৃথিবীর কোনো server-এর সাথে কথা হচ্ছে না — সমুদ্রের নিচের কোনো cable-এ তোমার প্রশ্ন যাচ্ছে না — এই বুদ্ধিটা এখন **তোমার hard disk-এর একটা file**, তোমার RAM-এ বসে ভাবছে।

এই অনুভূতিটা একটু উপভোগ করো। ওই ২ GB-র file-টার ভেতরে মানবজাতির লেখালেখির একটা সংকুচিত ছায়া বসে আছে — আর সেটা এখন তোমার পকেটের laptop-এ।

(চ্যাট থেকে বেরোতে লিখো `/bye`, অথবা `Ctrl+D`। আর internet-টা আবার চালু করে নিও — বইয়ের বাকি অংশে লাগবে।)

সততার প্যারা: local AI-এর সীমাবদ্ধতা

উত্তেজনার পাশাপাশি সত্যিটাও বলি — এই B model-টা Groq-এর B-র সমান না, হওয়ার কথাও না। কয়েকটা জিনিস খেয়াল করবে:

- **গতি:** GPU ছাড়া ৮ GB machine-এ উত্তর আসবে ধীরে-সুস্থে — পড়ার গতিতে, Groq-এর মতো ঝপ করে না। এটাই স্বাভাবিক।
- **বাংলা দুর্বল:** ছোট model-গুলোর বাংলা প্রায়ই ভাঙা-ভাঙা, মাঝে মাঝে হাস্যকর ভুলও করবে। English-এ কিন্তু এরাই বেশ ভালো। বাংলা নিয়ে serious কাজে তাই Groq-এর বড় model-ই ভরসা — এটা মেনে নিয়েই আগাব।
- **বুদ্ধি:** জটিল যুক্তি, লম্বা নির্দেশনা মানা — এসবে ছোট model হোঁচট খায়। ছোটখাটো কাজে সে চ্যাম্পিয়ন, PhD-র প্রশ্নে না।
- **Machine-এর অবস্থা:** চলার সময় fan ঘুরবে, machine একটু গরম হবে, অন্য app ধীর লাগতে পারে। আগের অধ্যায়ের পরামর্শ মনে করো — browser-এর tab-বাহিনী বন্ধ রাখো। আর computer পুরো hang হয়ে গেলে বুঝবে model-টা তোমার RAM-এর জন্য বড় — ছোটটাতে নামো (Appendix-এও এ নিয়ে টুকিটাকি আছে)।

তাহলে ছোট model দিয়ে লাভ কী? লাভটা পরের section-এই পরিষ্কার হবে — কারণ কিছু কাজে local model শুধু "চলে" না, রীতিমতো **সেরা পছন্দ।**

Python থেকে Ollama: অধ্যায় ৩-এর সেই promise

মনে আছে, অধ্যায় ৩-এ "এক চার্জারে সব ফোন" গল্পে বলেছিলাম — OpenAI-compatible শেখার লাভ পরে বুঝবে? সেই "পরে"-টা এখন।

Ollama install হওয়ার সাথে সাথে সে তোমার computer-এর ভেতরেই একটা ছোট API server চালিয়ে রাখে — ঠিকানা: `http://localhost:` (localhost মানে "এই computer-টাই")। আর সেই server-টা **OpenAI-compatible!** মানে?

মানে তোমার `chatbot.py` -কে Groq থেকে local AI-তে আনতে বদলাতে হবে **শুধু ঠিকানা আর model-এর নাম:**

Licensed to sakilorb@gmail.com · Do not redistribute

```
client = OpenAI(
    api_key="ollama",         Ollama-র key লাগে না — ঘরের জিনিস, তবে ঘরটা ফাঁকা রাখা যায় না বলে যেকোনো লেখা
দিলেই হয়
    base_url="http://localhost:          /v   "
)
```

আর model-এর জায়গায়:

```
        model="llama   .   ",
```

ব্যস। বাকি **সব — system prompt, messages খাতা, while loop, trimming — হুবহু একই।** এক শেখাতেই cloud আর local, দুই জগতের দরজা খোলা — এটাই standard শেখার শক্তি।

(খেয়াল করো — এখানে `.env` -র `GROQ_API_KEY` -ও লাগছে না, internet-ও লাগছে না। তোমার প্রশ্নগুলো computer ছেড়ে কোথাও যাচ্ছেই না।)

Local vs Groq: কখন কোনটা?

দুটো ইঞ্জিনই এখন তোমার garage-এ। কোন সফরে কোনটা বের করবে?

পরিস্থিতি	বেছে নাও	কেন
গোপনীয় তথ্য নিয়ে কাজ (ব্যক্তিগত note, client-এর data)	**Ollama**	তথ্য computer-এর বাইরে যায়ই না — এর চেয়ে বড় privacy হয় না
Internet নেই / অতি ধীর	**Ollama**	Offline-ই তো এর superpower
শত শত বার চালিয়ে experiment (যেমন loop-এ ৫০০টা call)	**Ollama**	কোনো rate limit নেই, কোনো দৈনিক হিসাব নেই — চালাও যত খুশি
বাংলায় ভালো মানের উত্তর দরকার	**Groq**	বড় model-এর বাংলা বহুগুণ শক্তিশালী
জটিল যুক্তি / agent-এর কঠিন সিদ্ধান্ত	**Groq**	B-র বুদ্ধির কাছে B শিশু
গতি চাই, এক্ষুনি	**Groq**	LPU-র ঝড়ের সাথে ঘরের CPU পারবে না

সহজ থাম্ব-রুল: **শেখা, খেলা, গোপন জিনিস, সীমাহীন experiment → local। মান, বাংলা, জটিল কাজ → Groq।** সামনের অধ্যায়গুলোতে আমরা মূলত Groq দিয়েই আগাব (agent-এর বুদ্ধি জরুরি বলে), কিন্তু প্রতিটা code-ই তুমি চাইলে দুই লাইন বদলে local-এ চালাতে পারবে — সেই স্বাধীনতাটা এখন তোমার।

Licensed to sakilorb@gmail.com · Do not redistribute

নিজে করো ৬

. `chatbot.py` -র একটা copy বানাও, নাম দাও `local_chatbot.py` । ওপরের দুটো বদল করো (base_url + model)। চালাও — তোমার "বন্ধু" এখন সম্পূর্ণ তোমার computer-এ থাকে!

. **আসল পরীক্ষা:** internet বন্ধ করে `local_chatbot.py` চালাও। কাজ করছে? অভিনন্দন — এই জিনিস দুনিয়ার খুব কম মানুষ নিজের হাতে বানিয়েছে।

. **তুলনা করো:** একই প্রশ্ন (একটা বাংলায়, একটা English-এ) দুই chatbot-কে করো — `chatbot.py` (Groq) আর `local_chatbot.py` (Ollama)। উত্তরের মান, গতি, বাংলার দক্ষতা — পার্থক্যগুলো নিজের চোখে দেখো। ওপরের table-টা তখন আর মুখস্থ করতে হবে না, ভেতর থেকে বুঝে যাবে।

. **Bonus:** `ollama list` লিখে দেখো তোমার computer-এ কী কী model আছে। আরেকটা ছোট model নামিয়ে (`ollama pull qwen . : b`) দুটোর তুলনা করতে পারো — জায়গা ফুরালে `ollama rm model_নাম` দিয়ে মুছে ফেলা যায়।

তোমার হাতে এখন দুই জগতের চাবি — cloud-এর দৈত্য আর পকেটের জ্বিন। কিন্তু দুটোই এখনো অধ্যায় ১-এর "রফিক" — কথার ওস্তাদ, কাজের বেলায় শূন্য। পরের অধ্যায়ে সেই সীমানাটা ভাঙব: LLM-কে দেব তার প্রথম **হাত-পা।**

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৭: Tool Calling — LLM-কে হাত-পা দেওয়া

জ্ঞানীর দুর্বলতা, নিজের চোখে

শুরুতেই একটা মজা করি। তোমার `chatbot.py` (বা Playground) খুলে এই দুটো প্রশ্ন করো:

. "এখন ঠিক কয়টা বাজে?"

. "৪৮৭৩ গুণ ৯২৬১ কত? শুধু সংখ্যাটা বলো।"

প্রথম প্রশ্নে সে হয় ভদ্রভাবে বলবে "আমার পক্ষে জানা সম্ভব না", নয়তো — আরও মজার — আত্মবিশ্বাসের সাথে একটা ভুল সময় বলে দেবে। দ্বিতীয়টায় সে একটা উত্তর দেবে, দেখতে বিশ্বাসযোগ্য — কিন্তু calculator-এ মিলিয়ে দেখো, প্রায়ই ভুল। (সঠিক উত্তর: ৪,৫১,২৮,৮৫৩ — আর হ্যাঁ, এটা আমি calculator দিয়েই বের করেছি; মুখে মুখে এই গুণ করার সাহস আমারও নেই।)

কেন এমন হয়? অধ্যায় ১-এ ফিরে যাও: LLM হলো **পরের শব্দ আন্দাজ করার যন্ত্র।** তার কাছে ঘড়ি নেই — "কয়টা বাজে"-র উত্তরে সে শুধু আন্দাজ করে এরকম প্রশ্নের পরে সাধারণত কেমন উত্তর বসে। আর বড় গুণের বেলায় সে হিসাব করে না, উত্তরটা কেমন দেখতে হতে পারে সেটা আন্দাজ করে। ছোট গুণ (৭×৮) সে লক্ষবার দেখেছে বলে পারে; ৪৮৭৩×৯২৬১ জীবনে দেখেনি, তাই বানিয়ে বলে।

আমাদের ঘরবন্দি জ্ঞানীর এটাই সীমা — জ্ঞান অগাধ, কিন্তু ঘরে ঘড়িও নেই, calculator-ও নেই। আজ আমরা তাঁকে সেগুলো দেব না — তার চেয়ে চালাক একটা কাজ করব।

বুদ্ধিটা: চিরকুট ব্যবস্থার নতুন নিয়ম

জ্ঞানীর ঘরে ঢোকা যায় না, কিন্তু চিরকুট তো চলেই। তাহলে তাঁকে যদি বলে দিই:

> "শুনুন — আপনার হয়ে কাজ করার জন্য বাইরে আমি (মানে আমার Python code) দাঁড়িয়ে আছি। আমি দুটো কাজ পারি: ঘড়ি দেখা আর calculator চালানো। কোনোটা দরকার হলে সরাসরি উত্তর না দিয়ে চিরকুটে লিখে দিন — 'ঘড়ি দেখে এসো' বা 'এই হিসাবটা করে আনো'। আমি করে এনে ফলাফল জানাব, তারপর আপনি চূড়ান্ত উত্তর দেবেন।"

ব্যস — এটাই **tool calling!** পুরো ব্যাপারটা চারটা ধাপের নাচ:

```
১. তুমি → LLM:    প্রশ্ন + "এই এই tool আছে" তালিকা
২. LLM → তুমি:    "আমার [tool]-টা চালাও, এই এই তথ্য দিয়ে"
৩. তুমি:           function-টা চালালে → ফলাফল LLM-কে পাঠালে
৪. LLM → তুমি:    ফলাফল দেখে সাজানো চূড়ান্ত উত্তর
```

আর এখান থেকেই এই বইয়ের সবচেয়ে গুরুত্বপূর্ণ সত্যটা বেরিয়ে আসে — এত জরুরি যে box-এ বাঁধিয়ে রাখছি:

Licensed to sakilorb@gmail.com · Do not redistribute

যে ভুল ধারণাটা এখনই ভাঙতে হবে

LLM নিজে কখনো কোনো tool চালায় না। চালায় তোমার code।

LLM শুধু অনুরোধ লেখে — "এই function-টা, এই তথ্য দিয়ে চালাও।" সেই অনুরোধ পড়ে function-টা চালানো, ফলাফল ফেরত দেওয়া — পুরোটাই তোমার Python code-এর কাজ। LLM-এর হাত-পা বলতে যা বোঝাচ্ছি, সেই হাত-পা আসলে **তোমার লেখা code।**

এটা বোঝা কেন এত জরুরি? কারণ এর মানে — নিয়ন্ত্রণ পুরোটাই তোমার হাতে। LLM চাইলেই তোমার computer-এ কিছু করে ফেলতে পারে না; শুধু সেটুকুই সম্ভব, যেটুকু করার function তুমি লিখে দিয়েছ। দরজার চাবি তোমার পকেটে।

ধাপ ১: Tools বানাই — এরা নেহাতই Python function

প্রথমে অবাক করা খবরটা: tool মানে বিশেষ কিছুই না — **সাধারণ Python function।** `ai-agent` folder-এ নতুন file খোলো — `tools.py` — আর শুরু করো:

```
import os
import json
from datetime import datetime
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()

client = OpenAI(
    api_key=os.getenv("GROQ_API_KEY"),
    base_url="https://api.groq.com/openai/v  "
)

    ---------- আমাদের দুটো tool: সাধারণ Python function ----------

def get_current_time():
    """এখনকার তারিখ ও সময় ফেরত দেয়"""
    return datetime.now().strftime("%d-%m-%Y, %I:%M %p")

def calculator(expression):
    """গাণিতিক রাশির হিসাব করে, যেমন: '                         '"""
            নিরাপত্তা: শুধু সংখ্যা আর অঙ্কের চিহ্ন ঢুকতে দেব, অন্য কিছু না
    allowed = set("                      +-  /().% ")
    if not set(expression) <= allowed:
        return "শুধু সংখ্যা এবং + -       / ( ) চিহ্ন ব্যবহার করা যাবে"
    try:
        return str(eval(expression))
    except Exception as e:
        return f"হিসাবে সমস্যা হয়েছে: {e}"
```


Licensed to sakilorb@gmail.com · Do not redistribute

(`eval` জিনিসটা যেকোনো Python code চালিয়ে ফেলতে পারে বলে এমনিতে বিপজ্জনক — তাই ওপরে `allowed` দিয়ে দরজায় পাহারা বসিয়েছি: সংখ্যা আর অঙ্কের চিহ্ন ছাড়া কিছু ঢুকলেই ফেরত। ছোট কিন্তু জরুরি অভ্যাস।)

ধাপ ২: LLM-কে tool-গুলো চেনাই — schema

Function তো বানালাম, কিন্তু LLM জানবে কী করে এগুলোর অস্তিত্ব আছে? তাকে একটা **তালিকা** পাঠাতে হবে — প্রতিটা tool-এর নাম, কাজের বর্ণনা, আর কী কী তথ্য লাগে। এই বর্ণনাপত্রের নাম **schema**, লেখা হয় একটা নির্দিষ্ট ছকে। `tools.py` -তে যোগ করো:

```
    ---------- LLM-এর জন্য tool-দের পরিচয়পত্র (schema) ----------

tools = [
    {
        "type": "function",
        "function": {
            "name": "get_current_time",
            "description": "বর্তমান তারিখ ও সময় জানায়। ব্যবহারকারী সময়, তারিখ বা 'কয়টা বাজে' জানতে চাইলে এটা ব্যবহার করো।",
            "parameters": {"type": "object", "properties": {}, "required": []}
        }
    },
    {
        "type": "function",
        "function": {
            "name": "calculator",
            "description": "গাণিতিক হিসাব নির্ভুলভাবে করে। যেকোনো যোগ, বিয়োগ, গুণ, ভাগের দরকারে নিজে আন্দাজ না করে এটা ব্যবহার করো।",
            "parameters": {
                "type": "object",
                "properties": {
                    "expression": {
                        "type": "string",
                        "description": "হিসাবের রাশি Python ছকে, যেমন: '
        ' বা '(                ) +      '"
                    }
                },
                "required": ["expression"]
            }
        }
    }
]

    নাম দিয়ে আসল function খুঁজে পাওয়ার টেবিল
AVAILABLE_FUNCTIONS = {
    "get_current_time": get_current_time,
    "calculator": calculator,
}
```


Licensed to sakilorb@gmail.com · Do not redistribute

ছকটা প্রথম দেখায় খটমট লাগে, কিন্তু ভেঙে দেখো — প্রতিটা schema আসলে তিনটা প্রশ্নের উত্তর: **নাম কী** (`name`), **কী কাজ করে আর কখন ব্যবহার করবে** (`description`), **কী কী তথ্য লাগে** (`parameters`)। `get_current_time` -এর কিছুই লাগে না বলে তার `properties` ফাঁকা; `calculator` -এর লাগে একটা `expression` ।

এর মধ্যে `description` **-টাই আসল জাদুর জায়গা।** LLM শুধু এই লেখাটুকু পড়েই সিদ্ধান্ত নেয় কোন প্রশ্নে কোন tool ডাকবে — এ ছাড়া সে তোমার function-এর ভেতরের এক লাইন code-ও দেখে না। তাই description যত পরিষ্কার, tool-বাছাই তত নিখুঁত। খেয়াল করো calculator-এর description-এ লিখেছি "নিজে আন্দাজ না করে এটা ব্যবহার করো" — এটা ইচ্ছাকৃত; LLM-এর গুণ-আন্দাজ-করার বদভ্যাসটা আমরা এখানেই আটকে দিলাম। চিনতে পারছ? **এটাও আসলে prompt engineering** — কন্টেক্সট ইঞ্জিনিয়ারিং বইয়ের সেই পুরোনো শিক্ষা, নতুন পোশাকে।

ধাপ ৩: পুরো নাচটা code-এ

এবার চারটা ধাপের নাচটা সাজাই। `tools.py` -র শেষে যোগ করো:

Licensed to sakilorb@gmail.com · Do not redistribute

```
    ---------- এবার পুরো round-trip ----------

messages = [
    {"role": "system", "content": "তুমি একজন সহায়ক বাংলাভাষী assistant। দরকারমতো tool ব্যবহার করে নির্ভুল উত্তর দেবে।"},
    {"role": "user", "content": "এখন কয়টা বাজে? আর শোনো — ৪৮৭৩ গুণ ৯২৬১ কত?"}
]

    ধাপ ১-২: প্রশ্ন + tool-তালিকা পাঠাই, LLM-এর অনুরোধ আসে
response = client.chat.completions.create(
    model="llama-  .  -    b-versatile",
    messages=messages,
    tools=tools                          ← নতুন জিনিস শুধু এটুকুই!
)

msg = response.choices[  ].message

if msg.tool_calls:
            LLM উত্তর দেয়নি — কাজের অনুরোধ পাঠিয়েছে!
            অনুরোধটা আগে খাতায় (messages-এ) তুলে রাখি
    messages.append(msg)

            ধাপ ৩: প্রতিটা অনুরোধ ধরে ধরে function চালাই
    for tool_call in msg.tool_calls:
        fn_name = tool_call.function.name
        fn_args = json.loads(tool_call.function.arguments)        লেখা থেকে dict-এ

        print(f"   LLM-এর অনুরোধ: {fn_name}({fn_args})")

        result = AVAILABLE_FUNCTIONS[fn_name](    fn_args)          ← আসল কাজ এখানে!

                    ফলাফলটা খাতায় লিখি — নতুন role: "tool"
        messages.append({
            "role": "tool",
            "tool_call_id": tool_call.id,          কোন অনুরোধের উত্তর, তার ঠিকানা
            "content": result
        })

            ধাপ ৪: ফলাফলসহ খাতা আবার পাঠাই — এবার আসে চূড়ান্ত উত্তর
    final = client.chat.completions.create(
        model="llama-  .  -    b-versatile",
        messages=messages,
        tools=tools
    )
    print("\n  ", final.choices[  ].message.content)
else:
            Tool লাগেনি — সাধারণ উত্তর
    print("  ", msg.content)
```

চালাও — `python tools.py`। Screen-এ দেখবে অনেকটা এরকম:

Licensed to sakilorb@gmail.com · Do not redistribute

```
    LLM-এর অনুরোধ: get_current_time({})
    LLM-এর অনুরোধ: calculator({'expression': '                    '})

  এখন সময় ২২-০৭-২০২৬, বিকাল ০৪:৩০। আর ৪৮৭৩ × ৯২৬১ = ৪,৫১,২৮,৮৫৩।
```

দেখো কাণ্ডটা — **একই LLM, যে দশ মিনিট আগে সময় জানত না আর গুণে ভুল করছিল, এখন দুটোই নির্ভুল বলে দিচ্ছে।** তার বুদ্ধি বাড়েনি এক ফোঁটাও; বেড়েছে তার নাগাল — তোমার দেওয়া হাত-পায়ের কল্যাণে।

Code-এর সূক্ষ্ম জায়গাগুলো, ঝেড়েকেশে

- `msg.tool_calls` — LLM যখন tool চায়, তখন সে সাধারণ লেখা-উত্তরের বদলে এই বিশেষ ঘরে অনুরোধ(গুলো) পাঠায়। খেয়াল করো — গুলো: আমাদের এক প্রশ্নে দুটো কাজ ছিল, LLM চালাকি করে **দুটো অনুরোধ একসাথেই** পাঠিয়েছে, তাই `for` loop।
- `json.loads(...)` — LLM-এর পাঠানো অনুরোধে function-এর তথ্যগুলো (arguments) আসে লেখা (string) আকারে: `'{"expression": "                    "}'`। এই লেখার ছকটার নাম JSON। `json.loads` সেটাকে Python-এর dict বানিয়ে দেয়, যেন `fn_args` দিয়ে function-এ ঢেলে দিতে পারি।
- `messages.append(msg)` — সূক্ষ্ম কিন্তু অত্যাবশ্যক লাইন! LLM-এর অনুরোধটাও কথোপকথনের অংশ — খাতায় না তুললে পরের ধাপে LLM নিজেই ভুলে যাবে সে কী চেয়েছিল, আর error আসবে।
- `role: "tool"` — অধ্যায় ৪-এর তিন role-এর সংসারে চতুর্থ সদস্য: tool-এর ফলাফলের নিজস্ব আসন। সাথে `tool_call_id` — একাধিক অনুরোধ থাকলে কোন ফলাফল কোন অনুরোধের, LLM এটা দিয়েই মেলায়।
- **LLM-ই ঠিক করে tool লাগবে কি না** — "কেমন আছো?" জিজ্ঞেস করলে সে কোনো tool না ডেকে সোজা উত্তর দেবে (আমাদের `else` অংশ)। কখন হাত-পা লাগবে, কখন মুখই যথেষ্ট — এই বিচারটুকুই তার বুদ্ধির পরিচয়।

আর সবচেয়ে বড় কথা — box-টার কথাটা আরেকবার মেলাও: পুরো নাচে LLM শুধু চিঠি লিখেছে। ঘড়ি দেখেছে কে? তোমার `datetime`। গুণ করেছে কে? তোমার `eval`। **হাত-পা তোমার code-ই।**

Licensed to sakilorb@gmail.com · Do not redistribute

নিজে করো ৭

. `tools.py` -র `user` প্রশ্নটা বদলে শুধু একটা কাজের প্রশ্ন করো (যেমন শুধু সময়), তারপর tool-ই লাগে না এমন প্রশ্ন করো ("বাংলাদেশের রাজধানী কী?")। প্রতিবার লাইনগুলো দেখো — LLM কখন কোনটা ডাকছে, কখন ডাকছেই না।

. **মূল challenge — নিজের প্রথম tool বানাও।** তিন জায়গায় হাত দিতে হবে: function লেখো → schema-য় পরিচয়পত্র যোগ করো → `AVAILABLE_FUNCTIONS` -এ নাম তোলো। Idea লাগলে: `roll_dice()` (লুডুর ছক্কা — `random.randint( , )`), `bmi_calculator(weight_kg, height_m)` , বা `count_letters(word, letter)` — শেষেরটা দিয়ে বিখ্যাত "strawberry-তে কয়টা r?" প্রশ্নে LLM-কে এবার নির্ভুল বানিয়ে ছাড়ো!

. **Bonus:** নিজের tool-এর `description` ইচ্ছা করে ঘোলাটে করে দাও (শুধু লেখো "একটা কাজ করে"), তারপর দেখো LLM ঠিক সময়ে সেটা ডাকতে পারে কি না। Description-এর ক্ষমতাটা নিজের চোখে দেখার এর চেয়ে ভালো উপায় নেই।

এবার হিসাবটা মেলাও: অধ্যায় ১-এর সূত্র ছিল **Agent = LLM + Tools + Loop**। LLM (অধ্যায় ২-৬)। Tools (এই মাত্র)। বাকি রইল শুধু **Loop** — আর ওই শেষ টুকরোটা জুড়লেই রফিক হয়ে যাবে সুমন। পরের অধ্যায়: আসল AI Agent।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৮: আসল AI Agent — Think, Act, Observe, Repeat

সেই মুহূর্তটা এসে গেছে

অধ্যায় ১-এ একটা কথা দিয়েছিলাম — বইয়ের শেষ দিকে তুমি নিজের হাতে সুমন বানাবে। আজ সেই দিন। এই অধ্যায় শেষে তোমার computer-এ এমন একটা program চলবে, যেটাকে তুমি বাংলায় একটা লক্ষ্য বলে দেবে, আর সে **নিজে ভেবে, নিজে সিদ্ধান্ত নিয়ে, ধাপে ধাপে** কাজটা শেষ করবে — তোমার চোখের সামনে, প্রতিটা ধাপ দেখিয়ে দেখিয়ে।

আর সবচেয়ে সুন্দর খবরটা? আগের অধ্যায়ের পর তোমার নতুন করে শেখার আছে **মাত্র একটা জিনিস।**

নতুন কর্মচারীর প্রথম দিন

সুমনের গল্পটা আরেকবার, এবার একটু ভেতর থেকে। দোকানদার তাকে লক্ষ্য দিলেন — "মাল এনে তাক সাজিয়ে হিসাব তুলে রাখো।" সুমনের মাথার ভেতর তারপর যা চলল:

```
ভাবল  →  "আগে গোডাউনে গিয়ে দেখি কী এসেছে"
করল   →  গোডাউনে গেল
দেখল  →  চাল-ডাল এসেছে, চিনি আসেনি
ভাবল  →  "আচ্ছা, চাল-ডাল দিয়েই শুরু করি"
করল   →  তাক সাজাল
দেখল  →  সাজানো শেষ
ভাবল  →  "এবার হিসাবের খাতা"
...
ভাবল  →  "সব শেষ। দোকানদারকে জানিয়ে দিই।"
```

লক্ষ করো — একই চারটা জিনিস ঘুরেফিরে আসছে: **ভাবো → করো → দেখো → আবার ভাবো।** Think → Act → Observe → Repeat। কাজ শেষ না হওয়া পর্যন্ত এই চাকাটা ঘুরতেই থাকে।

এবার আগের অধ্যায়ের সাথে মেলাও। Tool calling-এ কী করেছিলাম? LLM **ভাবল** (tool-এর অনুরোধ পাঠাল) → আমরা **করলাম** (function চালালাম) → LLM ফলাফল **দেখল** → চূড়ান্ত উত্তর দিল। মানে... এক পাক তো ঘোরাই হয়ে গেছে! শুধু আমরা এক পাকের পরেই থেমে গিয়েছিলাম।

তাহলে agent বানানোর "নতুন জিনিস"-টা কী? **থামব না।** LLM যতক্ষণ tool চাইতে থাকবে, আমরা চালাতে থাকব — ফলাফল ফেরত দিতে থাকব — আবার জিজ্ঞেস করতে থাকব "এরপর?"। পুরোটাকে একটা loop-এ মুড়ে দেওয়া — ব্যস, **এটাই agent!** সূত্রের শেষ টুকরো — Loop — এতই সরল।

আর কাজ শেষ হয়েছে বুঝব কীভাবে? উল্টো দিক থেকে: **LLM যেই পাকে আর কোনো tool চাইবে না, শুধু লেখা-উত্তর দেবে — সেটাই তার "কাজ শেষ, রিপোর্ট দিচ্ছি" signal!** সুমন যেমন সব সেরে দোকানদারের সামনে এসে মুখে বলে — "হয়ে গেছে।"

Licensed to sakilorb@gmail.com · Do not redistribute

আজকের project: ফাইল-গোছানো Agent

যে agent বানাব, তার কাজ: তোমার একটা folder-এর **সব file নিজে নিজে পড়ে, প্রতিটার summary বানিয়ে, সবকিছু গুছিয়ে একটা report file লিখে দেওয়া।** ভাবো — সেমিস্টারের ছড়ানো-ছিটানো note, মিটিং-এর টুকরো লেখা, idea-র file-এর স্তূপ — এক আদেশে সব পড়া হয়ে গোছানো report হাজির।

Agent-এর হাত-পা লাগবে তিনটা — তিনটাই file নিয়ে:

Tool	কাজ	সুমনের ভাষায়
`list_files`	folder-এ কী কী file আছে দেখা	গোডাউনে উঁকি দেওয়া
`read_file`	একটা file-এর লেখা পড়া	বস্তা খুলে দেখা
`write_file`	নতুন file লেখা	হিসাবের খাতায় তোলা

আগে মঞ্চ সাজাও: `ai-agent` folder-এর ভেতরে `notes` নামে একটা folder বানাও, আর তাতে নিজের মতো লেখা দিয়ে ২-৩টা ছোট file রাখো — যেমন `class-note.txt` (কোনো পড়ার টুকরো note), `todo.txt` (কাজের তালিকা), `idea.txt` (মাথায় ঘোরা কোনো idea)। যা খুশি লেখো, কয়েক লাইন হলেই চলবে — পড়বে তো agent, তুমি না।

Code, পর্ব ১: হাত-পা তিনটা

নতুন file: `agent.py`। প্রথম অংশ:

Licensed to sakilorb@gmail.com · Do not redistribute

```
import os
import json
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()

client = OpenAI(
    api_key=os.getenv("GROQ_API_KEY"),
    base_url="https://api.groq.com/openai/v  "
)

TARGET_FOLDER = "notes"           agent শুধু এই folder-এই কাজ করবে

    ---------- Agent-এর তিনটা হাত-পা ----------

def safe_path(filename):
    """নিরাপত্তা-পাহারা: folder-এর বাইরে যাওয়ার চেষ্টা আটকায়"""
    if ".." in filename or filename.startswith(("/", "\\")):
        return None
    return os.path.join(TARGET_FOLDER, filename)

def list_files():
    """Folder-এর সব file-এর নাম দেয়"""
    try:
        files = os.listdir(TARGET_FOLDER)
        return "\n".join(files) if files else "Folder-টা খালি।"
    except FileNotFoundError:
        return f"'{TARGET_FOLDER}' নামে folder পাওয়া যায়নি।"

def read_file(filename):
    """একটা file-এর লেখা পড়ে ফেরত দেয়"""
    path = safe_path(filename)
    if path is None:
        return "শুধু notes folder-এর ভেতরের file পড়া যাবে।"
    try:
        with open(path, "r", encoding="utf-  ") as f:
            return f.read()[:        ]           খুব বড় file হলে প্রথম অংশটুকু
    except FileNotFoundError:
        return f"'{filename}' নামে file নেই। আগে list_files দিয়ে সঠিক নাম দেখে নাও।"
    except Exception as e:
        return f"পড়তে সমস্যা: {e}"

def write_file(filename, content):
    """নতুন file বানিয়ে লেখা save করে"""
    path = safe_path(filename)
    if path is None:
        return "শুধু notes folder-এর ভেতরে লেখা যাবে।"
    try:
        with open(path, "w", encoding="utf-  ") as f:
            f.write(content)
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
        return f"'{filename}' লেখা হয়ে গেছে ({len(content)} অক্ষর)।"
    except Exception as e:
        return f"লিখতে সমস্যা: {e}"
```

দুটো জিনিসে চোখ রাখো:

- `safe_path` — আগের অধ্যায়ের সেই কথাটা মনে আছে? "শুধু সেটুকুই সম্ভব, যেটুকু তুমি লিখে দিয়েছ।" এই ছোট্ট পাহারাদার নিশ্চিত করছে agent-এর নাগাল `notes` folder-এর বাইরে এক পা-ও না যায়। Agent-কে ক্ষমতা দেওয়ার সাথে সাথে **সীমানা** দেওয়া — এই অভ্যাসটা প্রথম agent থেকেই রক্তে ঢুকিয়ে নাও।
- **Error message-গুলো দেখো** — `read_file` ভুল নামে ডাকা হলে শুধু "নেই" বলছে না, বলছে "আগে list_files দিয়ে সঠিক নাম দেখে নাও।" কেন? কারণ এই লেখাটা পড়বে **agent নিজে** — আর পথ-দেখানো নির্দেশনা পেলে সে নিজের ভুল নিজেই শুধরে নেয়! মানুষের জন্য error message লেখো ভদ্রভাবে, agent-এর জন্য লেখো রাস্তা দেখিয়ে।

Code, পর্ব ২: পরিচয়পত্র (schema)

আগের অধ্যায়ের চেনা ছক — নতুন কিছুই নেই, শুধু এবার তিনটা:

Licensed to sakilorb@gmail.com · Do not redistribute

```
    ---------- Tool-দের পরিচয়পত্র ----------

tools = [
    {
        "type": "function",
        "function": {
            "name": "list_files",
            "description": "কাজের folder-এ কী কী file আছে তার তালিকা দেয়। কাজ শুরুর আগে এটা দিয়েই দেখে নাও
কী কী আছে।",
            "parameters": {"type": "object", "properties": {}, "required": []}
        }
    },
    {
        "type": "function",
        "function": {
            "name": "read_file",
            "description": "নির্দিষ্ট একটা file-এর ভেতরের লেখা পড়ে ফেরত দেয়।",
            "parameters": {
                "type": "object",
                "properties": {
                    "filename": {"type": "string", "description": "File-এর নাম, যেমন
'todo.txt'"}
                },
                "required": ["filename"]
            }
        }
    },
    {
        "type": "function",
        "function": {
            "name": "write_file",
            "description": "নতুন একটা file বানিয়ে তাতে লেখা save করে।",
            "parameters": {
                "type": "object",
                "properties": {
                    "filename": {"type": "string", "description": "নতুন file-এর নাম, যেমন
'report.md'"},
                    "content": {"type": "string", "description": "File-এ যা লেখা হবে"}
                },
                "required": ["filename", "content"]
            }
        }
    }
]

AVAILABLE_FUNCTIONS = {
    "list_files": list_files,
    "read_file": read_file,
    "write_file": write_file,
}
```


Licensed to sakilorb@gmail.com · Do not redistribute

Code, পর্ব ৩: হৃৎপিণ্ড — The Agent Loop

এবার সেই অংশ, যার জন্য পুরো বইটা লেখা। মন দিয়ে দেখো — মোটে ৩৫ লাইনের মতো, আর প্রায় প্রতিটা লাইন তুমি আগে থেকেই চেনো:

Licensed to sakilorb@gmail.com · Do not redistribute

```
    ---------- Agent-এর মগজের নিয়মকানুন ----------

SYSTEM_PROMPT = """তুমি একজন দক্ষ AI Agent। তোমাকে একটা লক্ষ্য দেওয়া হবে।
তুমি ধাপে ধাপে ভাববে এবং প্রয়োজনমতো tool ব্যবহার করে লক্ষ্যটা পূরণ করবে।

নিয়ম:
- একসাথে সব না — এক ধাপ নাও, ফলাফল দেখো, তারপর পরের সিদ্ধান্ত নাও।
- কোনো তথ্য আন্দাজ করবে না — file-এ কী আছে জানতে হলে পড়ে দেখবে।
- পুরো লক্ষ্য পূরণ হয়ে গেলে আর কোনো tool ডাকবে না — তখন সংক্ষেপে
 বাংলায় জানাবে কী কী করলে।"""

    ---------- হৃৎপিণ্ড: The Agent Loop ----------

def run_agent(goal, max_steps=    ):
    messages = [
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": goal}
    ]

    for step in range(  , max_steps +    ):

                        THINK: পুরো খাতা পাঠিয়ে জিজ্ঞেস করি — "এরপর কী?"
        response = client.chat.completions.create(
            model="llama-  .  -    b-versatile",
            messages=messages,
            tools=tools
        )
        msg = response.choices[  ].message

                        Tool চায়নি = কাজ শেষ! এটাই থামার signal
        if not msg.tool_calls:
            print(f"\n   Agent-এর চূড়ান্ত রিপোর্ট:\n\n{msg.content}")
            return

        messages.append(msg)           অনুরোধটা খাতায় (আগের অধ্যায়ের সেই জরুরি লাইন!)

        for tool_call in msg.tool_calls:
            fn_name = tool_call.function.name
            fn_args = json.loads(tool_call.function.arguments)

                                ACT: অনুরোধমতো কাজটা করি
            print(f"[ধাপ {step}]    {fn_name}({fn_args})")
            result = AVAILABLE_FUNCTIONS[fn_name](    fn_args)

                                OBSERVE: ফলাফল খাতায় — পরের পাকে LLM এটাই দেখবে
            print(f"            {result[:    ]}")
            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "content": result
            })
```


Licensed to sakilorb@gmail.com · Do not redistribute

```
                    REPEAT: loop ঘুরে আবার THINK-এ


if __name__ == "__main__":
    run_agent(
        "notes folder-এর প্রতিটা file পড়ো। প্রতিটার ২-৩ লাইনের বাংলা summary বানাও। "
        "তারপর সব summary সুন্দরভাবে সাজিয়ে 'report.md' নামে একটা file-এ লিখে রাখো। "
        "(report.md file-টা নিজে আবার পড়তে যেও না।)"
    )
```

খেয়াল করেছ? — একটা জিনিস বাদ পড়ে গেছে মনে হচ্ছে। `for` loop যদি `max_steps` ধাপ শেষেও `return` না পায়? Function-টার একদম শেষে (loop-এর বাইরে, `if __name__` -এর আগে) এই লাইনটা বসাও:

```
    print(f"\n   {max_steps} ধাপেও কাজ শেষ হয়নি — নিরাপত্তার খাতিরে থামিয়ে দিলাম।")
```

এটা কেন এত জরুরি — একটু পরেই বলছি। আগে চালিয়ে দেখি!

Loop-এর চারটা মন্তব্য ( ) মিলিয়ে দেখো — **Think → Act → Observe → Repeat** আক্ষরিক অর্থে code হয়ে বসে গেছে।

চালাও — আর দৃশ্যটা উপভোগ করো

```
python agent.py
```

Screen-এ একটা একটা করে ভেসে উঠবে (তোমার file অনুযায়ী একটু আলাদা হবে):

```
[ধাপ    ]     list_files({})
                  class-note.txt
todo.txt
idea.txt
[ধাপ    ]     read_file({'filename': 'class-note.txt'})
                  আজকের class-এ recursion পড়ানো হলো। মূল কথা হলো — যে fun
[ধাপ    ]     read_file({'filename': 'todo.txt'})
                  ১. Assignment জমা দিতে হবে বৃহস্পতিবারের মধ্যে ২. মামার
[ধাপ    ]     read_file({'filename': 'idea.txt'})
                  একটা app বানানো যায় যেটা এলাকার টিউশনি খোঁজা সহজ করে
[ধাপ    ]     write_file({'filename': 'report.md', 'content': '    Notes Summ
                  'report.md' লেখা হয়ে গেছে (৬৪২ অক্ষর)।

   Agent-এর চূড়ান্ত রিপোর্ট:

notes folder-এর তিনটা file পড়ে প্রতিটার সংক্ষিপ্ত summary তৈরি করেছি
এবং সবগুলো গুছিয়ে report.md নামের file-এ লিখে রেখেছি।
```


Licensed to sakilorb@gmail.com · Do not redistribute

এবার `notes` folder খুলে দেখো — `report.md` **সত্যিই বসে আছে**, ভেতরে তোমার file-গুলোর গোছানো summary।

এক মিনিট থামো, ব্যাপারটার ওজনটা মাপো। **কোন file আগে পড়বে — তুমি বলোনি। কয়বার `read_file` ডাকতে হবে — তুমি বলোনি। কখন লেখা শুরু, কখন থামা — কিচ্ছু বলোনি।** তুমি দিয়েছ শুধু লক্ষ্য; প্রতিটা সিদ্ধান্ত সে নিয়েছে নিজে — তোমার লেখা loop-এর ভেতরে, তোমার দেওয়া হাত-পা দিয়ে। কাল `notes` -এ দশটা file রাখলে সে দশটাই পড়বে — code-এর একটা অক্ষরও বদলাতে হবে না।

অধ্যায় ১-এ যে দৃশ্যটা "শুধু কল্পনায়" দেখিয়েছিলাম — এইমাত্র সেটা ঘটল তোমার terminal-এ, তোমার নিজের বানানো code-এ। **রফিক আজ সুমন হলো।**

Agent কোথায় হোঁচট খায় — আর `max_steps` কেন জান বাঁচায়

সত্যিকারের agent বানিয়েছ মানে সত্যিকারের সমস্যাগুলোও চেনা দরকার। কয়েকবার চালালেই নিজে দু-একটা দেখে ফেলবে:

- **চক্করে আটকে যাওয়া।** কখনো agent একই file বারবার পড়তে থাকে, বা "শেষ করি করি" করেও tool ডেকেই চলে। এজন্যই `max_steps` — লাগাম। এটা ছাড়া agent অনন্তকাল ঘুরতে পারত, আর মনে রেখো — **প্রতিটা পাক মানে একটা API call।** লাগামছাড়া loop মানে দৈনিক limit গলগল করে খরচ। তাই agent-জগতের প্রথম নিরাপত্তা-সূত্র: থামার ব্যবস্থা আগে, চালু পরে। আমাদের goal-এর শেষে "report.md নিজে পড়তে যেও না" লাইনটাও খেয়াল করেছ? ওটাও এক দফা চক্কর-খাওয়া দেখেই বসানো — নাহলে কোনো কোনো agent report লিখে, তারপর সেটাই আবার পড়ে, তারপর ভাবে নতুন file পেলাম, আবার summary… বুঝতেই পারছ।
- **ভুল নাম আন্দাজ করা।** কখনো সে `note.txt` পড়তে চাইবে, যেখানে file-এর নাম আসলে `class-note.txt` । আর তখনই দেখবে পথ-দেখানো error message-এর জাদু — "আগে list_files দিয়ে দেখে নাও" পড়ে সে নিজেই list_files ডাকবে, সঠিক নাম নেবে, আবার চেষ্টা করবে। **নিজের ভুল নিজে সারানো** — এই দৃশ্যটা প্রথমবার দেখা রীতিমতো আনন্দের ব্যাপার।
- **নির্দেশ আধখানা মানা।** কখনো summary English-এ লিখে ফেলবে, কখনো ফরম্যাট এলোমেলো। ওষুধ দুটো: লক্ষ্যটা আরও স্পষ্ট করে লেখা (এখানেই কন্টেক্সট ইঞ্জিনিয়ারিংয়ের হাত পাকানো কাজে দেয়), আর জটিল কাজে বুদ্ধিমান model বাছা। কৌতূহল হলে এই agent-টাই Ollama-র B দিয়ে চালিয়ে দেখো (অধ্যায় ৬-এর দুই লাইনের বদল!) — দেখবে ছোট মগজ ঘনঘন পথ হারায়। অধ্যায় ৬-এর কথাটা হাতেনাতে: **agent-এর মগজ যত ভালো, পথ তত সোজা।**

Licensed to sakilorb@gmail.com · Do not redistribute

নিজে করো ৮

. `notes` -এ আরও ২টা file যোগ করে আবার চালাও — agent নিজেই মানিয়ে নিচ্ছে, দেখো।

. লক্ষ্য বদলাও: "সব file পড়ে যেগুলোতে করণীয় কাজ (todo) আছে খুঁজে বের করো, আর সব কাজ এক জায়গায় জড়ো করে 'sob-kaj.md' বানাও।" — একই agent, নতুন লক্ষ্য, code অপরিবর্তিত। এটাই agent-এর আসল শক্তি।

. **চতুর্থ হাত যোগ করো:** `delete_file` … না থাক — ওই ক্ষমতা দেওয়ার আগে agent আরেকটু বড় হোক। বরং `count_words(filename)` বানাও (file-এর শব্দসংখ্যা গোনে — hint: `len(text.split())`)। তিন-জায়গার checklist মনে আছে তো? — function → schema → `AVAILABLE_FUNCTIONS`। তারপর লক্ষ্য দাও: "কোন file-এ সবচেয়ে বেশি শব্দ, বের করে জানাও।" — নতুন হাতটা সে ঠিকঠাক ব্যবহার করছে কি না দেখো।

. **Bonus:** `max_steps=` করে ৫টা file-ওয়ালা folder-এ চালাও। লাগামে টান পড়াটা নিজের চোখে দেখো — আর বুঝে নাও কেন এই সাদামাটা লাইনটাই আসলে পুরো agent-এর সবচেয়ে দায়িত্ববান লাইন।

তোমার agent এখন জীবন্ত — কিন্তু সে জাগে কেবল তখনই, যখন তুমি নিজ হাতে `python agent.py` চালাও। শেষ প্রশ্নটা তাই নিজে থেকেই চলে আসে: **তুমি ঘুমিয়ে থাকলেও যদি সে কাজ করে যেত?** পরের অধ্যায়ে সেই দরজাটা খুলে দেব।

Licensed to sakilorb@gmail.com · Do not redistribute

অধ্যায় ৯: Agent যখন ঘুমায় না — ২৪/৭ চালানোর দরজা

একটা অতৃপ্তি রয়ে গেছে, তাই না?

তোমার agent এখন দিব্যি কাজ করে। কিন্তু একটা ব্যাপার খচখচ করছে নিশ্চয়ই — সে কাজ করে **কেবল তখনই, যখন তুমি নিজে হাতে `python agent.py` চালাও।** Laptop-এর ঢাকনা নামালে সে ঘুম; তুমি ভুলে গেলে সে-ও ভোলা।

অথচ স্বপ্নটা তো অন্যরকম ছিল, তাই না? প্রতিদিন সকাল ৮টায় agent নিজে জেগে উঠবে, কাজটা সেরে রাখবে — তুমি তখনো ঘুমে। সত্যিকারের automation বলতে আমরা এটাই বুঝি।

এই অধ্যায়ে আমরা সেই জগতে বসবাস করতে যাব না — সেটা আস্ত আরেকটা বইয়ের (আসলে আরেকটা course-এর) বিষয়। আজ শুধু দরজাটা খুলে ভেতরের দৃশ্যটা দেখাব — যেন পথটা আর অচেনা না থাকে। স্বপ্নটা পূরণ হতে লাগবে মোটে দুটো জিনিস।

জিনিস ১: একটা অ্যালার্ম-ঘড়ি (cron)

প্রথম প্রশ্ন — program-কে "প্রতিদিন সকাল ৮টায় নিজে চালু হও" বলব কীভাবে?

মজার ব্যাপার হলো, Linux আর Mac-এ এই অ্যালার্ম-ঘড়িটা **আগে থেকেই বসানো আছে** — নাম **cron**। কয়েক দশকের পুরোনো, পাথরের মতো নির্ভরযোগ্য এক ব্যবস্থা: তুমি একটা তালিকায় (নাম crontab) লিখে রাখবে কখন, কোন command চালাতে হবে — cron ঘড়ি ধরে ঠিক সেই সময়ে চালিয়ে দেবে। প্রতিদিন। কামাই নেই।

একটা crontab-এর লাইন দেখতে যেমন হয়:

```
cd /home/tomar-নাম/ai-agent && venv/bin/python agent.py
```

শুরুর পাঁচটা ঘর হলো সময়ের ঠিকানা — **মিনিট, ঘণ্টা, মাসের দিন, মাস, সপ্তাহের দিন।** এখানে মানে: ৮টা বেজে ০ মিনিটে, প্রতিদিন (মানে "যেকোনো")। আর বাকিটুকু তোমার চেনা — folder-এ ঢুকে agent চালানোর command।

আরও দুটো নমুনা, ছন্দটা ধরার জন্য:

```
/                                  → প্রতি ৩০ মিনিট পরপর
                                     → প্রতি শুক্রবার রাত ১০টায়
```

প্রথম দেখায় সংকেত-সংকেত লাগে, কিন্তু একবার ছন্দটা ধরলে রীতিমতো নেশা লেগে যায় — "নিজে করো"-তে এটা নিয়ে খেলার ব্যবস্থা আছে। (Windows-এ এই কাজটা করে **Task Scheduler** নামের একটা built-in tool — ধারণা একই, চেহারা click-ভিত্তিক।)

Licensed to sakilorb@gmail.com · Do not redistribute

জিনিস ২: যে computer ঘুমায় না (server)

অ্যালার্ম তো দিলাম, কিন্তু ঘড়িটা বাজবে কোথায়? তোমার laptop-এ cron বসালেও laptop বন্ধ থাকলে কিছুই চলবে না। দরকার এমন একটা computer, যেটা **কখনো বন্ধ হয় না।**

এই জিনিসটারই নাম **server**। আর নামটা শুনে মাথায় যে বিশাল, রহস্যময়, আলো-জ্বলা machine-এর ছবি আসে — সেটা এখনই ভেঙে দিই:

> **Server আসলে নেহাতই একটা computer, যেটা কেউ বন্ধ করে না। ব্যস। এটুকুই।**

চাইলে আজই তোমার একটা server হতে পারে, দুই রাস্তায়:

রাস্তা ১ — ভাড়া করা (VPS): পৃথিবীজুড়ে অনেক company-র বিশাল data center-এ হাজার হাজার computer দিনরাত চলছে; মাসে কয়েকশো টাকার বিনিময়ে তার একটা টুকরো তুমি ভাড়া নিতে পারো — একে বলে **VPS** (Virtual Private Server)। ভাড়া নেওয়ার পর নিজের ঘরে বসে internet-এর মাধ্যমে তাতে ঢুকবে (পদ্ধতিটার নাম SSH), তোমার `agent.py` আর crontab বসিয়ে দেবে — ব্যস। তারপর তোমার laptop বন্ধ, বাসার বিদ্যুৎ চলে গেছে, তুমি কক্সবাজারে — agent তবু প্রতিদিন সকাল ৮টায় জেগে নিজের কাজ করে যাচ্ছে।

রাস্তা ২ — ঘরের কোণার সৈনিক (free!): বাসার সেই পুরোনো laptop-টা — যেটা "ধীর হয়ে গেছে" বলে কোণায় পড়ে আছে? তাতে Linux বসাও, WiFi-এ জুড়ে দাও, চার্জারে লাগিয়ে কোণায় রেখে দাও — অভিনন্দন, তোমার একটা home server হলো, খরচ শূন্য টাকা। এই বইয়ের চেতনার সাথে বেশ মানানসই একটা রাস্তা, তাই না?

আর এখানেই অধ্যায় ৫-এর সেই কথাটা পূর্ণ বৃত্তে ফেরে। VPS হোক বা ঘরের কোণার laptop — **server-দুনিয়ার প্রায় পুরোটাই চলে Linux-এ।** সুখবরটা হলো: server-এ ঢুকে তুমি যা করবে — folder-এ ঢোকা, file চালানো, crontab লেখা — সবই সেই terminal-এর কারবার, যার সাথে এই বইয়ে তোমার হাতেখড়ি হয়েই গেছে। ভিতটাকে এবার পাকা দালান বানাতে চাইলে আমার Linux বইটা তোমার পরের স্বাভাবিক ধাপ — terminal-কে রীতিমতো পোষা বেড়াল বানিয়ে ছাড়বে: **লিনুকে পোষ মানানো** → https://iftesamulohy.com/products/linuke-posh-manano/

ছাড়ার আগে: লাগামের কথাটা আবার

Agent-কে ২৪/৭-এর স্বাধীনতা দেওয়ার আগে অধ্যায় ৮-এর শিক্ষাটা দ্বিগুণ জরুরি হয়ে ওঠে — কারণ এবার ভুলটাও চলবে ২৪/৭! ছাড়ার আগের ছোট্ট checklist:

- `max_steps` আছে তো? (দিনে ৩০ বার চলা agent × লাগামছাড়া loop = এক সকালে পুরো দৈনিক limit সাবাড়)
- Agent-এর নাগাল কি সীমানায় বাঁধা? (`safe_path` -এর কথা মনে করো)
- কোথাও একটা log রাখছ তো, যেন পরদিন দেখতে পারো রাতে সে কী কী করল?

স্বাধীনতা আর লাগাম — দুটো একসাথেই দিতে হয়। এটুকু মাথায় থাকলে তুমি ready।

Licensed to sakilorb@gmail.com · Do not redistribute

দরজার ওপারে: automation-এর জগৎ

এই অধ্যায়ে যা দেখালাম তা trailer মাত্র। পুরো ছবিটা — মানে সত্যিকারের automation জগৎটা — আরও অনেক বড়: server ঠিকঠাক সাজানো, agent-এর log রাখা, error হলে নিজেকে খবর পাঠানো, browser খুলে নিজে নিজে website-এ কাজ করা, একাধিক automation-কে সামলানো... এগুলো এক অধ্যায়ের জিনিস না, systematically শেখার জিনিস।

সুখবর: ঠিক এই জিনিসগুলো নিয়েই আমার একটা **recorded Automation Course** আসছে **এই আগস্টে** — হাতে-কলমে, বাংলায়, একদম গোড়া থেকে। এই বইটা যদি তোমার ভালো লেগে থাকে, course-টা তোমার জন্যই বানানো। আমার Facebook page-এ চোখ রাখো — ঘোষণা ওখানেই আগে আসবে।

নিজে করো ৯

. Browser-এ যাও **crontab.guru** — cron-এর সময়-ঠিকানা শেখার সবচেয়ে মজার জায়গা। ঘরগুলোতে সংখ্যা বসাও, সাথে সাথে সাদা বাংলায় (আচ্ছা, সাদা English-এ) মানে দেখাবে। Challenge: এই তিনটা বানিয়ে দেখাও — প্রতিদিন দুপুর ২টা, প্রতি রবিবার সকাল ৯টা, প্রতি মাসের ১ তারিখ রাত ১২টা।

. এবার ভাবো — **তোমার** agent-এর জন্য আদর্শ schedule কী? অধ্যায় ১-এর "নিজে করো"-তে যে কাজটা লিখে রেখেছিলে, সেটা দিনে কয়বার, কখন চললে জীবন সহজ হয়? Cron-এর ভাষায় লিখে ফেলো — খাতার সেই পাতাতেই।

. **Bonus (Linux/Mac):** সাহস থাকলে নিজের computer-এই ছোট্ট পরীক্ষা — terminal-এ `crontab -e` লিখে editor খোলো, একদম শেষে যোগ করো: `date >> /tmp/porikkha.txt` — save করে ২-৩ মিনিট অপেক্ষা, তারপর `cat /tmp/porikkha.txt` । প্রতি মিনিটে একটা করে সময়ের ছাপ জমছে দেখছ? ওই যে — **তোমার computer-এ প্রথমবার কিছু একটা চলল, তোমার হাত ছাড়াই।** দেখা শেষে `crontab -e` দিয়ে লাইনটা মুছে দিও।

ব্যস — hands-on যাত্রা এখানেই শেষ। বাকি রইল শুধু পেছন ফিরে দেখা: কতদূর এলে, আর সামনের রাস্তাগুলো কোনদিকে গেছে।

Licensed to sakilorb@gmail.com · Do not redistribute

শেষ অধ্যায়: এরপর কী?

একবার পেছন ফিরে দেখো

বইয়ের প্রথম পাতায় তুমি ছিলে সেই মানুষ, যে রাত এগারোটায় ফোন scroll করতে করতে ভাবছিল — "এসব করতে নিশ্চয়ই অনেক টাকা লাগে।"

আর আজ? আজ তোমার computer-এ:

- একটা **chatbot** চলে — বাংলায় কথা বলে, আগের কথা মনে রাখে, আর তার প্রতিটা লাইন তুমি নিজে লিখেছ, বুঝে লিখেছ
- একটা **local AI** বসে আছে — internet-এর তার খুলে ফেললেও যে কথা বলে যায়
- LLM-কে **হাত-পা দেওয়ার** কৌশল তোমার জানা — tool calling, schema, round-trip — এবং তুমি জানো সবচেয়ে গোপন সত্যটা: tool চালায় তোমার code, LLM না
- আর সবার ওপরে — একটা **আসল AI Agent**, যে নিজে ভেবে, নিজে সিদ্ধান্ত নিয়ে, ধাপে ধাপে কাজ শেষ করে — তোমার বানানো loop-এর ভেতরে, তোমার দেওয়া সীমানার মধ্যে
- এবং মোট খরচ, কথা রেখেছি তো? — **শূন্য টাকা।**

Think → Act → Observe → Repeat — এই চারটা শব্দ এখন তোমার কাছে আর জাদুমন্ত্র না, ৩৫ লাইনের চেনা code। আর যে জিনিস একবার ভেতর থেকে বুঝে ফেলেছ, সেটা নিয়ে hype-এর ঝড়ে তোমাকে আর কেউ বিভ্রান্ত করতে পারবে না। খবরে যখন পড়বে "অমুক company-র AI Agent এই করেছে, সেই করেছে" — তুমি মুচকি হেসে ভাববে: জানি ভাই। LLM + Tools + Loop। আমারও একটা আছে।

সামনের রাস্তাগুলো (নাম চিনে রাখো)

তোমার agent এখন শিশু — হাঁটতে শিখেছে মাত্র। সামনে তার বেড়ে ওঠার যে রাস্তাগুলো আছে, সেগুলোর নাম চিনিয়ে দিই — যেন সামনে যখন এই শব্দগুলো দেখবে, অচেনা না লাগে:

- **আরও শক্তিশালী tools:** আজ agent-এর হাত-পা file পর্যন্ত। কাল সেই হাত পৌঁছাতে পারে internet-এ (web search tool), database-এ, email-এ, তোমার নিজের যেকোনো Python function-এ। মনে রেখো — নতুন tool মানেই সেই তিন-জায়গার checklist; কাঠামোটা তুমি শিখেই ফেলেছ, এখন শুধু হাতের সংখ্যা বাড়ানোর ব্যাপার।
- **Memory আর RAG:** আমাদের agent প্রতিবার শূন্য থেকে শুরু করে। বড় agent-রা আগের কাজ মনে রাখে, আর দরকারমতো বিশাল document-ভাণ্ডার থেকে সঠিক তথ্য খুঁজে এনে পড়ে — এই খোঁজার কৌশলটার নাম RAG (Retrieval-Augmented Generation)। অধ্যায় ৪-এর সেই "খাতা ছেঁটে ফেলা"-র সমস্যাটার এরাই বড়সড় সমাধান।
- **Multi-agent:** এক সুমনে কুলোচ্ছে না? একাধিক agent — একজন খোঁজে, একজন লেখে, একজন যাচাই করে — মিলে দল বেঁধে কাজ করে। দোকান থেকে তখন সেটা রীতিমতো company।

Licensed to sakilorb@gmail.com · Do not redistribute

- **Agent framework:** LangChain, CrewAI — এই নামগুলো শুনবে চারদিকে। এরা আসলে আমাদের `agent.py`-র loop-টাই, অনেক সাজসরঞ্জামসহ প্যাকেট করা। ভালো খবর: ভেতরের কলকব্জা তুমি খালি হাতে বানিয়েছ বলে framework-এ ঢুকলে তোমার লাগবে অর্ধেক সময় — আর জাদু দেখে ভড়কাবে না, কারণ জাদুটা তুমি নিজেই জানো।

তোমার পরের তিনটা সঙ্গী

পুরো বইয়ে যাদের কথা প্রসঙ্গক্রমে এসেছে, এক জায়গায় গুছিয়ে রাখলাম — যার যেটা দরকার:

তোমার যদি মনে হয়...	তাহলে	কোথায়
"Python-এর ভিতটা আরও মজবুত চাই"	**Journey by Python** — শূন্য থেকে, বাংলায়, ধাপে ধাপে	https://iftesamulohy.com/products/journey-by-python/
"Terminal আর Linux-টা এবার পুরোপুরি আয়ত্তে আনব"	**লিনুকে পোষ মানানো** — গল্পের ছলে Linux-এ হাতেখড়ি থেকে দক্ষতা	https://iftesamulohy.com/products/linuke-posh-manano/
"Automation-টা এবার systematically, গভীরে শিখব"	**Recorded Automation Course** — আসছে এই আগস্টে	আমার Facebook page-এ ঘোষণা

শেষ কথা — আর তোমার প্রথম নিজের agent

মনে আছে, অধ্যায় ১-এর "নিজে করো"-তে একটা কাজ লিখে রাখতে বলেছিলাম — তোমার জীবনের সেই বিরক্তিকর, বারবার-করা কাজটা, যেটা একটা agent-কে দিতে চাও? খাতার সেই পাতাটা এবার খোলো।

তখন ওটা ছিল স্বপ্ন। এখন ওটা একটা **specification**। পড়ো, আর নিজেকে জিজ্ঞেস করো — এই কাজটার জন্য আমার agent-এর কী কী হাত-পা লাগবে? কোন tool? কী তার description? লক্ষ্যটা কী ভাষায় লিখলে সে পথ হারাবে না? — দেখো, প্রশ্নগুলোর উত্তর তুমি এখন জানো।

`agent.py`-র একটা copy করো। Tool-গুলো বদলাও। লক্ষ্যটা লেখো। চালাও।

প্রথমবার হয়তো হোঁচট খাবে — খাক। অধ্যায় ৮-এ শিখেছ, হোঁচট খাওয়া agent-কে কীভাবে পথে আনতে হয়। দ্বিতীয়বার, তৃতীয়বার — তারপর একসময় সেই মুহূর্তটা আসবে: তোমার নিজের ভাবা, নিজের বানানো agent, তোমার নিজের জীবনের একটা সত্যিকারের কাজ সেরে ফেলল।

সেই মুহূর্তে মনে রেখো — শুরুটা হয়েছিল ৫ লাইনের একটা "হ্যালো" দিয়ে।

যাত্রা শুভ হোক।

Licensed to sakilorb@gmail.com · Do not redistribute

Licensed to sakilorb@gmail.com · Do not redistribute

Appendix: বিপদে পড়লে এই পাতা (Troubleshooting)

আটকে গেছ? আগে এখানে দেখো — এই বইয়ের পাঠকদের ৯০% সমস্যা এই কয়টার মধ্যেই পড়ে। প্রতিটার ছক এক: **লক্ষণ → কারণ → সমাধান।**

১. `pip install` কাজ করছে না

লক্ষণ: `pip install` দিলে লাল লাল লেখা, `Could not find a version...` বা SSL/timeout জাতীয় error।

সমাধান, একটার পর একটা চেষ্টা করো: - আগে দেখো `(venv)` চালু আছে কি না — না থাকলে activate করো (অধ্যায় ৩) - `pip` -এর বদলে লেখো: `python -m pip install openai python-dotenv` (Windows-এ প্রায়ই এতেই মিটে যায়) - Internet ধীর হলে timeout বাড়াও: `pip install --timeout openai` - SSL error এলে সাধারণত কারণ পুরোনো Python — python.org থেকে হালনাগাদ Python (. +) install করো - Office/ university-র WiFi-তে proxy থাকলে অন্য network-এ (যেমন mobile hotspot) চেষ্টা করো — package ছোট, বেশি MB যাবে না

২. Windows terminal-এ বাংলা ভেঙে যাচ্ছে

লক্ষণ: Chatbot-এর বাংলা উত্তরের জায়গায় প্রশ্নবোধক চিহ্ন, বাক্স, বা দুর্বোধ্য অক্ষর।

কারণ: পুরোনো Command Prompt-এর জন্মই বাংলা যুগের আগে — সে by default UTF- (বিশ্বের সব ভাষা লেখার ব্যবস্থা) বোঝে না।

সমাধান: - Code চালানোর আগে Command Prompt-এ লেখো: `chcp` — এটা terminal-কে UTF- -এ নিয়ে আসে - তাতেও না মিটলে সবচেয়ে ভালো সমাধান: Microsoft Store থেকে free **Windows Terminal** app-টা নামিয়ে সেটাতে কাজ করো — আধুনিক, UTF- নিজে থেকেই বোঝে, দেখতেও ভালো - আর Python নিজে বাংলা লিখতে/পড়তে গিয়ে `UnicodeDecodeError` দিলে দেখো file খোলায় `encoding="utf- "` আছে কি না — আমাদের `agent.py` -তে যেমন আছে

৩. Groq বলছে : `Too Many Requests` / `rate_limit_exceeded`

লক্ষণ: কাজ করতে করতে হঠাৎ error, ভেতরে বা `rate limit` কথাটা আছে।

কারণ: অধ্যায় ২-এর সেই সীমা — মিনিটের বা দিনের limit ছুঁয়ে ফেলেছ। ভয়ের কিছু না, টাকাও কাটেনি (কাটার উপায়ই নেই)।

সমাধান: - মিনিটের limit হলে: এক-দুই মিনিট চা-বিরতি, তারপর আবার - দিনের limit হলে: ছোট model-এ নামো (`llama- . - b-instant` — দৈনিক limit অনেক বেশি), অথবা Ollama-য় চালাও (অধ্যায় ৬ — limit-ই নেই), অথবা কাল আবার - বারবার হলে দেখো কোথাও লাগামছাড়া loop চলছে কি না — অধ্যায় ৮-এর `max_steps` -এর কথা

Licensed to sakilorb@gmail.com · Do not redistribute

মনে করো - Error-টা ভদ্রভাবে সামলাতে চাইলে API call-টা `try/except` -এ মুড়ে কিছুক্ষণ অপেক্ষা করে আবার চেষ্টার ব্যবস্থা করা যায় — automation course-এ এসব কায়দা গভীরে দেখাব

8. API key নিয়ে গোলমাল (`AuthenticationError` / / `Invalid API Key`)

লক্ষণ: Code চালালেই authentication জাতীয় error।

সমাধান-checklist, ওপর থেকে নিচে: - `.env` file-টা কি ঠিক **যে folder থেকে code চালাচ্ছ, সেখানেই** আছে? (সবচেয়ে common কারণ এটাই) - File-এর নাম কি ঠিক `.env` ? Windows-এ notepad অনেক সময় চুপিচুপি `.env.txt` বানিয়ে ফেলে — file explorer-এ "File name extensions" চালু করে দেখো - `.env` -এর ভেতরের লাইনটা কি ঠিক এই ছকে: `GROQ_API_KEY=gsk_...` — quotation নেই, `=` -এর দুপাশে space নেই? - Key copy করার সময় শুরু/শেষে space বা এক টুকরো কম copy হয়ে যায়নি তো? - Code-এ `load_dotenv()` লাইনটা আছে তো, এবং সেটা `os.getenv(...)` -এর আগে? - সব ঠিক, তবু error? — console.groq.com-এ ঢুকে পুরোনো key delete করে নতুন একটা বানাও, নতুনটা বসাও

৫. Ollama-র model download আটকে যাচ্ছে / বারবার ছিঁড়ছে

লক্ষণ: `ollama pull` চলতে চলতে থেমে থাকে, বা connection error-এ কেটে যায়।

সমাধান: - মনে রাখো: **আবার একই `ollama pull` command দাও** — যেখানে থেমেছিল মোটামুটি সেখান থেকেই ধরবে, শূন্য থেকে না। ছেঁড়া-জোড়া করে করেই বড় model নামানো যায় - গতি একেবারে মন্থর হলে অধ্যায় ৬-এর box-এর কৌশলগুলো: রাতে নামাও, ভালো WiFi-এর নাগালে সেরে রাখো - বারবারই ব্যর্থ হলে ছোট model দিয়ে শুরু করো (`ollama pull llama . : b` — মোটে ~১.৩ GB), বড়টা পরে সুবিধামতো - Disk-এ জায়গা আছে তো? Model-এর size + আরও ২-৩ GB খালি থাকা ভালো। পুরোনো model মুছতে: `ollama rm model-নাম`

৬. Ollama চালালে computer hang / আঠার মতো ধীর

লক্ষণ: `ollama run` দেওয়ার পর পুরো computer জমে যায়, fan-এ ঝড়, মাউস পর্যন্ত নড়ে না।

কারণ: অধ্যায় ৫-এর সেই "পড়ার টেবিল" — model-টা তোমার RAM-এর তুলনায় বড়, তাই মেঝেতে (disk-এ) বই রেখে পড়ার দশা।

সমাধান: - ছোট model-এ নামো — এটাই আসল সমাধান: `ollama run llama . : b` - চালানোর আগে RAM-খেকোদের বিদায় করো — browser-এর tab-বাহিনী সবার আগে - মাঝপথে আটকে গেলে: Ollama-র terminal-এ `Ctrl+C` ; পুরো system জমে গেলে ভয় না পেয়ে restart — ক্ষতি হয় না - আর মনে রেখো, এ যুদ্ধে হার মানারও দরকার নেই — Groq তো আছেই। বইয়ের প্রতিটা project cloud দিয়েই সম্পূর্ণ করা যায় (অধ্যায় ৫-এর সেই ভরসার কথা)

৭. Agent অদ্ভুত আচরণ করছে (ভুল tool, চক্কর, আধখানা কাজ)

লক্ষণ: Agent ঘুরপাক খাচ্ছে, ভুল file খুঁজছে, বা কাজ অর্ধেক করে "শেষ" ঘোষণা দিচ্ছে।

Licensed to sakilorb@gmail.com · Do not redistribute

সমাধান: - এটা আসলে error না — এটাই agent-জীবন! অধ্যায় ৮-এর "Agent কোথায় হোঁচট খায়" অংশটা আবার পড়ো — তিনটা রোগ আর ওষুধ ওখানেই - চটজলদি checklist: লক্ষ্যটা আরও স্পষ্ট লেখো → tool-এর `description` আরও পথ-দেখানো করো → জটিল কাজে বড় model (`llama-  .  -  b-versatile`) দাও → `max_steps` ঠিক আছে কি না দেখো - Debug-এর সেরা বন্ধু: `` আর `` লাইনগুলো মন দিয়ে পড়া — agent কী দেখে কী সিদ্ধান্ত নিচ্ছে পুরোটা তো তোমার চোখের সামনেই ছাপা হচ্ছে

আর কোনো সমস্যায় আটকে গেলে? Error-এর লেখাটুকু মন দিয়ে পড়ো (উত্তরের অর্ধেক ওখানেই থাকে), তারপর error message-টা ধরে internet-এ search করো — এই দক্ষতাটাই, সত্যি বলতে, একজন developer-এর সবচেয়ে বড় superpower। আর হ্যাঁ — চাইলে তোমার chatbot-কেও জিজ্ঞেস করে দেখতে পারো। সে এখন তোমারই বানানো, মনে আছে তো?

— সমাপ্ত —

"নিজের AI Agent হাতে-কলমে" · হাতে-কলমে সিরিজ ২ · Ifte Samul Ohy · iftesamulohy.com

Licensed to sakilorb@gmail.com · Do not redistribute